

**UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL MENDOZA
TECNICATURA UNIVERSITARIA EN
PROGRAMACIÓN**

**SIEM Inteligente con Machine Learning: Detección Automática
de Anomalías en Registros de Seguridad**

*Monitoreo y Detección: Análisis Automatizado de Registros y Detección de
Anomalías*

Trabajo Final de Técnico Universitario en Programación

Autor: Gianfranco Canciani

Directores de Tesis: Alberto Cortez y Ariel Enferrel

Línea de Investigación: Automatización en Ciberseguridad - Analítica de
Logs - SOAR

Mendoza – Argentina 2026

Resumen

La creciente complejidad de las amenazas informáticas demanda soluciones de seguridad que superen las limitaciones de la detección tradicional basada en reglas estáticas. Para responder a este desafío, esta investigación presenta el diseño, despliegue y validación empírica de un Sistema de Gestión de Información y Eventos de Seguridad (SIEM) de próxima generación, potenciado por Inteligencia Artificial. El núcleo innovador de esta plataforma radica en la implementación de un motor de Machine Learning no supervisado, el cual emplea los algoritmos Isolation Forest y DBSCAN para identificar proactivamente anomalías y patrones de ataque en entornos de red dinámicos, sin requerir etiquetado previo de datos.

Toda la arquitectura fue desarrollada bajo una filosofía de código abierto y bajo costo, estructurando un pipeline de ingesta y procesamiento masivo de logs heterogéneos mediante Syslog-ng, Logstash y Elasticsearch. Sobre esta base analítica, se integró un módulo de Orquestación, Automatización y Respuesta de Seguridad (SOAR) impulsado por n8n, el cual trasciende la mera notificación para ejecutar contramedidas defensivas de forma autónoma e interactiva. Este orquestador implementa el enriquecimiento de datos mediante Inteligencia sobre Amenazas (OSINT) y geolocalización en tiempo real. Además, introduce el paradigma *Human-in-the-Loop* a través de mensajería bidireccional (bots interactivos con botoneras de acción), permitiendo la ejecución de bloqueos activos de red a nivel de kernel vía SSH y despachando reportes ejecutivos matutinos de forma programada. El ecosistema se completa con paneles de visualización táctica a través de Grafana y un registro de auditoría inmutable en PostgreSQL, asegurando la trazabilidad absoluta de cada incidente.

Para validar la efectividad de la solución, el sistema fue sometido a pruebas de estrés dentro de un entorno académico simulado (compuesto por dos servidores y cinco hosts). Al contrastar el desempeño de este modelo predictivo y de mitigación activa frente a métodos convencionales de revisión manual, los resultados evidenciaron un salto cualitativo sustancial: se logró una reducción del 38 % en el Tiempo Medio de Detección (MTTD) y, de manera crítica, una disminución del 27 % en la tasa de falsos positivos. En definitiva, este proyecto demuestra empíricamente que es posible democratizar el acceso a infraestructuras defensivas interactivas y de alto rendimiento, ofreciendo una arquitectura altamente replicable, eficiente y escalable para laboratorios universitarios, centros de investigación y organizaciones con limitaciones presupuestarias.

Palabras clave: SIEM de próxima generación, Machine Learning, automatización SOAR, Inteligencia sobre Amenazas (OSINT), Human-in-the-Loop, n8n, Isolation Forest, código abierto.

Abstract

The growing complexity of cyber threats demands security solutions that overcome the limitations of traditional static rule-based detection. To address this challenge, this research presents the design, deployment, and empirical validation of a Next-Generation Security Information and Event Management (SIEM) system, powered by Artificial Intelligence. The innovative core of this platform lies in the implementation of an unsupervised Machine Learning engine, which employs the Isolation Forest and DBSCAN algorithms to proactively identify anomalies and attack patterns in dynamic network environments, without requiring prior data labeling.

The entire architecture was developed under an open-source and low-cost philosophy, structuring a pipeline for the mass ingestion and processing of heterogeneous logs using Syslog-ng, Logstash, and Elasticsearch. Upon this analytical foundation, an automated Security Orchestration, Automation, and Response (SOAR) module driven by n8n was integrated, which transcends mere notification to execute defensive countermeasures autonomously and interactively. This orchestrator implements data enrichment through Threat Intelligence (OSINT) feeds and real-time geolocation. Furthermore, it introduces the *Human-in-the-Loop* paradigm through bidirectional messaging (interactive bots with actionable buttons), enabling the execution of active network blocks at the kernel level via SSH and dispatching scheduled morning executive reports. The ecosystem is completed with real-time tactical visualization dashboards through Grafana and an immutable audit logging system in PostgreSQL, ensuring absolute traceability of every incident.

To validate the effectiveness of the solution, the system was subjected to stress testing within a simulated academic environment (consisting of two servers and five hosts). By comparing the performance of this predictive and active mitigation model against conventional manual review methods, the results evidenced a substantial qualitative leap: a 38% reduction in the Mean Time to Detect (MTTD) was achieved, alongside a critical 27% decrease in the false positive rate. Ultimately, this project empirically demonstrates that it is possible to democratize access to high-performance, interactive defensive infrastructures, offering a highly replicable, efficient, and scalable architecture for university laboratories, research centers, and organizations with budget constraints.

Keywords: Next-Generation SIEM, Machine Learning, SOAR automation, Threat Intelligence (OSINT), Human-in-the-Loop, n8n, Isolation Forest, open-source.

Agradecimientos

El autor expresa su más sincero agradecimiento a los docentes de las áreas de Seguridad Informática y de Programación de la Universidad Tecnológica Nacional (Alberto Cortez y Ariel Enferrel). Su orientación académica, experiencia técnica y constante disposición fueron pilares fundamentales para la conceptualización, el desarrollo técnico y la culminación exitosa de este proyecto de investigación.

Índice General

1. Introducción

- **1.1 Contexto del Problema**
- **1.2 Planteamiento del Problema**
- **1.3 Justificación**
- **1.4 Objetivo General**
- **1.5 Objetivos Específicos**
- **1.6 Alcance y Limitaciones**

2. Marco Teórico

- **2.1 Fundamentos de SIEM y Arquitecturas SOAR**
 - 2.1.1 El Ecosistema SIEM: Recolección y Análisis Centralizado
 - 2.1.2 Arquitecturas SOAR: La Automatización de la Respuesta
 - 2.1.3 Sinergia SIEM/SOAR: Hacia una Respuesta Activa
- **2.2 Evolución hacia UEBA y Next-Generation SIEM**
- **2.3 Justificación de los Modelos de Machine Learning No Supervisado**
- **2.4 Simulación de Amenazas bajo el Framework MITRE ATT&CK**

3. Marco Metodológico

- **3.1 Tipo y Nivel de Investigación**
- **3.2 Enfoque Metodológico**
- **3.3 Diseño de la Investigación**
- **3.4 Escenario Experimental y Unidad de Análisis**
- **3.5 Técnicas e Instrumentos de Recolección de Datos**
 - **3.5.1 Instrumentos de Captura y Procesamiento (Syslog-ng, Generador Sintético, Logstash/Elasticsearch)**
 - **3.5.2 Consideraciones Éticas y Tratamiento de Datos (Anonimización, Protocolo HITL e Integridad)**
 - **3.5.3 Disponibilidad de Datos y Artefactos Técnicos**
- **3.6 Procedimiento Experimental y Simulación de Amenazas**
- **3.7 Técnicas de Procesamiento y Análisis de Datos**

- **3.8 Amenazas a la Validez y Mitigación**

4. Desarrollo de la Investigación (Arquitectura e Implementación)

- **4.1 Arquitectura del Sistema e Implementación Tecnológica**
- **4.2 Componentes Principales del Pipeline de Datos**
- **4.3 Arquitectura de Seguridad Interna**
- **4.4 Lógica de Detección y Casos de Uso (Playbooks)**

5. Resultados y Análisis

- **5.1 Análisis de Métricas de Rendimiento Operativo**
- **5.2 Fiabilidad del Sistema y Análisis de Fallos**
- **5.3 Calibración de Algoritmos y Detección de Anomalías**
- **5.4 Discusión de Resultados**

6. Conclusiones y Trabajo Futuro

- **6.1 Conclusiones**
- **6.2 Trabajo Futuro**

7. Consideraciones Éticas y Recomendaciones

- **7.1 Principios Éticos y Tratamiento de Datos**
- **7.2 Recomendaciones Técnicas y Operativas**

8. Referencias

9. Anexos

- **Anexo A:** Cronograma de Implementación
- **Anexo B:** Esquema SQL de Auditoría (PostgreSQL)
- **Anexo C:** Configuración del Ingestor (Logstash)
- **Anexo D:** Catálogo de Reglas y Lógica de Orquestación
- **Anexo E:** Configuración del Bot de Telegram y SOAR en n8n
- **Anexo F:** Simulación de Infraestructura y Ejecución de Defensa Activa (SSH)
- **Anexo G:** Implementación de Respuesta Interactiva y Protocolo HITL (RU-4)
- **Anexo H:** Implementación del Módulo de Auditoría y Reportes Consolidados (Reporte Ejecutivo Matutino)
- **Anexo I:** Implementación del Motor de Machine Learning Real y Sistema de Registro Multiusuario

1. Introducción

Este capítulo presenta el marco fundacional de la investigación, proporcionando una visión general del escenario actual en el que se desarrolla el proyecto. A través de los siguientes apartados, se describe la problemática de seguridad que motiva este estudio, se formalizan los objetivos que guían el desarrollo del sistema y se delimitan los alcances técnicos y operativos de la solución propuesta.

1.1 Contexto del Problema

En el panorama contemporáneo de la ciberseguridad, la detección temprana y la mitigación ágil de incidentes se han convertido en requisitos fundamentales para garantizar la resiliencia de las infraestructuras tecnológicas. El crecimiento exponencial en la superficie de ataque y el volumen masivo de datos generados por los sistemas modernos exigen capacidades avanzadas para correlacionar eventos dispersos y responder con celeridad. La telemetría generada por servidores y dispositivos de red es tan vasta que el análisis puramente humano resulta insuficiente para identificar vectores de ataque complejos en tiempo real.

1.2 Planteamiento del Problema

A pesar de la importancia crítica del monitoreo, las soluciones SIEM (Security Information and Event Management) comerciales tradicionales suelen presentar barreras significativas de adopción. Los elevados costos de licenciamiento, la complejidad técnica de su despliegue y los altos requerimientos de hardware las vuelven prohibitivas para entornos de aprendizaje, laboratorios de investigación académica o pequeñas organizaciones con presupuestos limitados.

Esta situación genera una brecha de seguridad en instituciones que, aun manejando información valiosa, carecen de herramientas para trascender la detección basada en reglas estáticas, las cuales son ineficaces contra amenazas de "día cero" o comportamientos anómalos no catalogados. Ante este escenario, surge la siguiente pregunta de investigación:

¿Es posible diseñar y validar una arquitectura SIEM de código abierto que integre Machine Learning y automatización SOAR para reducir significativamente los tiempos de detección y respuesta ante incidentes en entornos con recursos limitados?

1.2.1 Hipótesis de Investigación

La implementación de un pipeline SIEM de código abierto, integrado con algoritmos de Machine Learning no supervisado (Isolation Forest y DBSCAN) y orquestación SOAR interactiva mediante n8n, reduce significativamente el Tiempo Medio de Detección (MTTD) y la tasa de falsos positivos en comparación con el análisis manual de registros, operando eficientemente en entornos de laboratorio con recursos limitados.

1.3 Justificación

La presente investigación se justifica en el valor técnico y metodológico que aporta el diseño de una arquitectura de defensa activa integrando capacidades UEBA en un entorno puramente open-source. El proyecto trasciende la monitorización clásica al implementar algoritmos de aprendizaje no supervisado (Isolation Forest y DBSCAN) para la identificación proactiva de amenazas complejas sin necesidad de etiquetado previo. Asimismo, la innovación operativa de este trabajo radica en la introducción del paradigma Human-in-the-Loop mediante orquestación interactiva, lo cual optimiza la toma de decisiones defensivas y establece un modelo técnico altamente replicable para futuras investigaciones en el área de SecOps.

1.4 Objetivo General

Diseñar y validar un pipeline tecnológico capaz de recolectar volúmenes masivos de registros (logs), detectar anomalías de seguridad mediante algoritmos de inteligencia artificial adaptativos, y orquestar respuestas automáticas de contención ante comportamientos sospechosos.

1.5 Objetivos Específicos

- **Implementar una arquitectura SIEM reproducible:** Utilizar componentes open-source bajo el paradigma de Infraestructura como Código (IaC) garantizando una tasa de disponibilidad del sistema durante las pruebas de estrés.
- **Integrar algoritmos de Machine Learning:** Aplicar modelos de aprendizaje no supervisado para la detección de anomalías, apuntando a alcanzar un F1-Score superior a 0.85 sin requerir un proceso de etiquetado manual.
- **Automatizar la respuesta frente a incidentes (SOAR):** Configurar el orquestador n8n para ejecutar ciclos de mitigación interactivos, logrando una tasa de respuestas exitosas (ejecución de contramedidas) superior al 90%.
- **Consolidar la visibilidad estratégica:** Desarrollar tableros de control en Grafana que permitan la inspección de métricas, reduciendo el Tiempo Medio de Detección (MTTD) en al menos un 25% respecto a la revisión analítica manual.

1.6 Alcance y Limitaciones

La investigación se centra en el diseño y despliegue de la arquitectura dentro de un entorno de red local simulado, compuesto por dos servidores y cinco hosts de prueba. El sistema está diseñado específicamente para ser escalable y replicable en entornos educativos y laboratorios académicos.

Como limitaciones principales, se reconoce que el uso de registros sintéticos para las pruebas de estrés iniciales puede no capturar toda la entropía de una red de producción a gran escala. Asimismo, la ejecución de bloqueos activos automáticos se limita a entornos Linux mediante comandos de kernel (*iptables*), sin contemplar en esta fase inicial integraciones con firewalls propietarios de capa de aplicación.

En este contexto, es imperativo establecer que la presente investigación se consolida formalmente como una Prueba de Concepto (PoC) académica. En consecuencia, si bien los

resultados cuantitativos y las mejoras operativas demostradas son estadísticamente significativas dentro del laboratorio controlado de dos servidores y cinco hosts, estas métricas actúan como una línea base demostrativa de la viabilidad de la arquitectura propuesta, y no pretenden establecer garantías deterministas de rendimiento en redes de producción empresarial masiva.

2. Marco Teórico

El presente capítulo establece los fundamentos conceptuales, normativos y tecnológicos que sustentan el desarrollo de esta investigación. Se analizan los paradigmas actuales en la gestión de eventos de seguridad, contrastando los enfoques deterministas clásicos con las estrategias heurísticas modernas. Asimismo, se justifica la adopción de algoritmos de Machine Learning bajo un enfoque de aprendizaje no supervisado y se describe el marco de referencia **MITRE ATT&CK** utilizado para la validación empírica del sistema.

2.1 Fundamentos de SIEM y Arquitecturas SOAR

El núcleo de las operaciones de seguridad modernas en un SOC (Security Operations Center) reside en la capacidad de transformar volúmenes masivos de datos crudos en inteligencia accionable. En un entorno donde la telemetría generada por servidores y dispositivos es tan vasta que el análisis humano resulta insuficiente para identificar vectores complejos en tiempo real, se requiere de una arquitectura que automatice el procesamiento de eventos. Esta sección analiza la evolución técnica de las dos tecnologías pilares que conforman la solución propuesta.

2.1.1 El Ecosistema SIEM: Recolección y Análisis Centralizado

Históricamente, los sistemas SIEM han evolucionado desde simples gestores de registros (Log Management) hacia plataformas de inteligencia distribuida. Según el estándar NIST SP 800-92, una gestión de registros eficaz es crítica para el cumplimiento normativo y la respuesta a incidentes. El funcionamiento de un SIEM moderno se divide en un ciclo de vida de cuatro etapas críticas:

- **Recolección y Agregación:** Consiste en la ingesta de registros provenientes de fuentes heterogéneas, utilizando protocolos como Syslog, SNMP o agentes Beats. El desafío técnico reside en el volumen; por ello, en este proyecto se utiliza **Syslog-ng** para permitir una pre-agregación eficiente en la capa de transporte, reduciendo la carga del motor de indexación.
- **Normalización y Parsing:** Es el proceso de convertir logs de diversos formatos (JSON, texto plano, CSV) en un esquema común. Esta etapa es vital para resolver el problema de la fragmentación de taxonomías. Al adoptar el **Elastic Common Schema (ECS)**, se logra que campos disparos como `src_ip`, `source_address` o

`ip_orig` sean tratados bajo una única identidad estandarizada. Esto facilita búsquedas transversales correlacionadas, permitiendo que un firewall y un servidor web "hablen el mismo idioma" técnico.

- **Enriquecimiento:** Proceso de añadir contexto a los registros en tiempo real, transformando un dato plano en información contextualizada. Esto incluye la geolocalización de IPs, resolución de nombres de dominio (DNS) o consultas a bases de datos de reputación (OSINT), lo que permite determinar instantáneamente si una conexión proviene de una región geográfica inusual o de un nodo conocido de una botnet.
- **Correlación y Alerta:** Aplicación de lógica programática para identificar incidentes. En sistemas tradicionales, esto se limita al análisis de firmas apoyadas en Indicadores de Compromiso (IoC) conocidos. Sin embargo, los IoCs son volátiles y caducan rápidamente. Esta limitación técnica frente a ataques polimórficos o de **Día Cero** justifica la transición hacia un modelo de detección basado en anomalías y comportamiento adaptativo.

2.1.2 Arquitecturas SOAR: La Automatización de la Respuesta

Para mitigar el desfase temporal entre la detección de una amenaza y su remediación, surge el paradigma SOAR (*Security Orchestration, Automation, and Response*). Mientras que el SIEM responde al "qué está pasando", el SOAR gestiona el "cómo reaccionar" de manera eficiente. Sus pilares se definen de la siguiente manera:

- **Orquestación:** Es la capacidad de interconectar herramientas de seguridad dispares (firewalls, bases de datos de amenazas, servicios de mensajería) a través de sus APIs para que trabajen como un ecosistema coordinado.
- **Automatización:** Se basa en la ejecución de **Playbooks** (flujos de trabajo estandarizados) sin intervención humana. Esto resuelve el fenómeno de la "**Fatiga de Alertas**" (*Alert Fatigue*), donde el exceso de falsos positivos abruma al analista. Al automatizar tareas de bajo nivel, como la verificación de una IP en VirusTotal, el analista puede centrarse en investigaciones de alta prioridad.
- **Respuesta:** Gestión integral del incidente. La adopción de **n8n** como orquestador permite definir flujos reactivos que contienen amenazas en tiempo real, aplicando el concepto de defensa activa y garantizando que las contramedidas se ejecuten en segundos.

2.1.3 Sinergia SIEM/SOAR: Hacia una Respuesta Activa y Resiliente

Al unificar un SIEM con capacidades SOAR, se logra una transición de un modelo de monitorización pasiva a un ecosistema de mitigación interactiva. Esta integración permite optimizar métricas de rendimiento críticas como el **MTTR** (*Mean Time to Respond*). En infraestructuras modernas de microservicios, donde un ataque puede propagarse lateralmente en milisegundos, la agilidad de la respuesta automatizada —como el bloqueo automático de una IP tras una detección de Machine Learning— es la única vía para garantizar la resiliencia operativa y la continuidad del negocio.

Estudios empíricos recientes en la gestión de incidentes han demostrado que la integración

de herramientas SOAR en centros de operaciones de seguridad reduce drásticamente el tiempo de mitigación. Como señalan Islam et al. (2023), la automatización de la recolección de contexto y la ejecución de playbooks predefinidos permite optimizar el MTTR hasta en un 40%, liberando carga cognitiva del analista ante alertas repetitivas.

2.1.4 Trabajos Relacionados y Casos de Uso Comparables

El diseño de infraestructuras defensivas de bajo costo ha sido objeto de diversas investigaciones recientes que sirven de cimiento para este trabajo, centrándose principalmente en la superación de las barreras económicas de las soluciones privativas. Una referencia fundamental es el trabajo de Manzoor et al. (2024), quienes en su investigación "Cybersecurity on a budget" evaluaron el rendimiento y la seguridad de soluciones SIEM de código abierto específicamente para pequeñas y medianas empresas (PYME). Este estudio demostró que es técnicamente viable alcanzar niveles de visibilidad robustos utilizando herramientas libres, siempre que se realice una correcta gestión y normalización de los registros de auditoría.

Paralelamente, la investigación académica ha evidenciado que la mayoría de las arquitecturas actuales presentan un vacío operativo crítico. Estudios contemporáneos sobre la gestión de inteligencia de amenazas, como los análisis de Tonhauser y Ristvej (2023) y Anthor et al. (2019), documentan que los flujos de trabajo en los centros de operaciones (SOC) suelen detenerse en la fase de detección pasiva y visualización. Según Schlette et al. (2021), aunque las plataformas SIEM generan alertas precisas, la remediación suele delegarse a procesos manuales, lo que introduce inconsistencias, incrementa la fatiga cognitiva en los analistas y genera retrasos inaceptables en el Tiempo Medio de Respuesta (MTTR).

La presente investigación se nutre de estos antecedentes para establecer su contribución original: propone cerrar el ciclo de vida del incidente mediante la integración directa de un módulo de Orquestación, Automatización y Respuesta de Seguridad (SOAR) impulsado por n8n. Al acoplar esta base analítica con la ejecución de bloqueos activos y bidireccionales, se transforma el sistema de un monitor pasivo en un sensor defensivo reactivo. Esta sinergia permite que la arquitectura no solo detecte la amenaza, sino que ejecute contramedidas en tiempo real (como el bloqueo de IPs vía SSH), unificando la detección y la mitigación en un único pipeline de código abierto de alta eficiencia, abordando directamente el vacío empírico identificado en la literatura reciente.

2.2 Evolución hacia UEBA y Next-Generation SIEM

La industria de la ciberseguridad experimenta una transición paradigmática, desplazándose de sistemas deterministas basados en firmas hacia plataformas avanzadas que incorporan análisis heurístico y de comportamiento. Este proyecto se alinea con la definición de **Next-Generation SIEM** (Sistemas de Gestión de Eventos e Información de Seguridad de Próxima Generación), los cuales trascienden la simple recolección de logs para integrar

nativamente capacidades de UEBA (User and Entity Behavior Analytics).

La arquitectura de un Next-Gen SIEM no solo centraliza la telemetría, sino que aplica analítica avanzada para detectar amenazas que operan "bajo el radar" de los controles tradicionales. Esta evolución se fundamenta en tres pilares críticos que se desarrollan en esta investigación:

- **De la Regla al Comportamiento:** A diferencia de las reglas de correlación clásicas, que operan bajo una lógica binaria y umbrales fijos (ej. "si hay 5 intentos fallidos, alertar"), UEBA construye perfiles dinámicos y líneas base (baselines) del comportamiento estándar de "entidades", término que engloba tanto a usuarios humanos como a dispositivos, cuentas de servicio y direcciones IP. Al establecer qué es lo "normal" en una red específica, el sistema puede identificar desviaciones sutiles que representan un riesgo potencial.
- **Análisis Multidimensional:** Mientras que los sistemas tradicionales analizan eventos de forma aislada, el enfoque UEBA permite una correlación multifuente. Esto significa que el sistema puede vincular un pico inusual de consumo de CPU (Recursos) con una conexión SSH fuera de horario (Tiempo) y una geolocalización anómala (Espacio), asignando un puntaje de riesgo integral. Esta capacidad es vital para mitigar la "fatiga de alertas".
- **Fundamentación Científica y Detección Estructural:** La lógica de detección de este proyecto se sustenta en la literatura científica. Investigaciones contemporáneas, como los estudios de Thakkar y Lohiya (2021) sobre sistemas de detección de intrusiones para entornos interconectados, concluyen que los modelos basados en aislamiento (Isolation Forest) y densidad (DBSCAN) resultan superiores en eficiencia computacional y precisión para identificar vectores de inyección masiva en arquitecturas de red modernas. Asimismo, la definición del mercado ha madurado: el informe *Magic Quadrant for Security Information and Event Management* (Davies et al., 2024), publicado por la firma Gartner, establece que las capacidades nativas de UEBA y orquestación SOAR ya no son complementos opcionales, sino requisitos fundamentales para clasificar a una plataforma como un SIEM de próxima generación.

Al integrar UEBA, este SIEM inteligente deja de ser una herramienta de auditoría pasiva para convertirse en un sensor proactivo capaz de modelar la incertidumbre inherente a los entornos de red dinámicos.

2.3 Justificación de los Modelos de Machine Learning No Supervisado

La selección de algoritmos representa el núcleo analítico de la investigación. En el contexto de la ciberseguridad contemporánea, las infraestructuras generan gigabytes de telemetría diaria. Debido a que en redes productivas los registros carecen de etiquetas previas que clasifiquen categóricamente el tráfico como "malicioso" o "legítimo", la implementación de modelos de aprendizaje supervisado resulta inviable

debido a los altos costos operativos y temporales asociados al etiquetado manual exhaustivo. Asimismo, un modelo supervisado quedaría rápidamente obsoleto frente a variaciones en las técnicas de ataque. Por ello, se optó por el Aprendizaje No Supervisado, seleccionando algoritmos capaces de inferir la estructura oculta de los datos, una decisión respaldada por investigaciones recientes como las de Thakkar y Lohiya (2021), quienes evidencian la superioridad operativa de los modelos basados en aislamiento y densidad para la detección de intrusiones en arquitecturas complejas.

1. Isolation Forest (Aislamiento): Este modelo se basa en la premisa estadística de que las anomalías son, por naturaleza, "pocas y diferentes". A diferencia de los métodos tradicionales que intentan perfilar exhaustivamente el comportamiento normal para luego buscar desviaciones, este algoritmo aísla activamente los valores atípicos. El algoritmo construye múltiples árboles de decisión aleatorios (iTrees) para aislar puntos de datos. La métrica fundamental aquí es la "longitud del camino" (path length): debido a que un evento malicioso es métricamente distante de las observaciones estándar, requiere un número significativamente menor de particiones aleatorias para ser aislado en una hoja del árbol. Por el contrario, un evento normal, al estar inmerso en un clúster denso, requerirá muchas más particiones. Su ventaja competitiva para este proyecto, tal como señalan Thakkar y Lohiya (2021) al evaluar modelos de Machine Learning para detección de intrusiones, radica en su baja complejidad computacional ($O(n \log n)$), lo que permite procesar miles de registros por segundo en tiempo real sin degradar el rendimiento del servidor de indexación subyacente.

Si bien el modelo fundacional de Liu, Ting y Zhou (2008) estableció las bases matemáticas del aislamiento espacial, investigaciones contemporáneas como las de Hariri, Kind y Brunner (2019) sobre Extended Isolation Forest han demostrado la necesidad de abordar las limitaciones de la partición ortogonal en conjuntos de datos de alta dimensionalidad. No obstante, tras un análisis comparativo, para el alcance de esta investigación y la topología de red simulada de un centro de datos universitario, se determinó que la versión clásica ofrece el equilibrio óptimo entre precisión y bajo consumo de recursos. Esta elección permite mantener la agilidad del pipeline sin requerir la sobrecarga computacional de modelos extendidos, garantizando la viabilidad técnica en hardware de gama media.

2. DBSCAN (Agrupamiento Espacial Basado en Densidad): Este modelo agrupa puntos que presentan una alta densidad temporal o espacial, clasificando como "ruido" a los elementos que quedan aislados en regiones de baja densidad. La potencia analítica de DBSCAN para entornos SIEM radica en la interacción geométrica de dos hiperparámetros configurables: la distancia máxima de vecindad (ϵ o epsilon) y el número mínimo de registros requeridos para conformar un clúster (min_samples). Esta capacidad paramétrica lo hace excepcionalmente eficaz para detectar ataques de fuerza bruta o escaneos horizontales de puertos. En estos

escenarios, el adversario genera una inyección agresiva y anómala de telemetría en una ventana de tiempo muy estrecha, elevando drásticamente la densidad local de los datos. DBSCAN detecta esta anomalía de densidad de forma orgánica, encapsulando el ataque sin necesidad de un entrenamiento previo con firmas de ataque y sin requerir que el administrador pre defina cuántos clústeres deben existir en la red. Esta capacidad de agrupar ataques masivos sin conocimiento previo es otra de las razones por las cuales la literatura especializada (Thakkar y Lohiya, 2021) destaca los algoritmos basados en densidad como complementos ideales a los modelos de aislamiento en infraestructuras de seguridad.

2.4 Simulación de Amenazas bajo el Framework MITRE ATT&CK

Para validar el sistema con rigor científico y representatividad real, se adoptó la matriz MITRE ATT&CK (Adversary Tactics, Techniques, and Common Knowledge) , un estándar global que documenta las tácticas, técnicas y procedimientos (TTPs) empleados por los adversarios modernos. A diferencia de modelos secuenciales y teóricos previos como la *Cyber Kill Chain*, el framework de MITRE proporciona una ontología empírica. Su estructura clasifica el comportamiento malicioso en **Tácticas**, que representan el "por qué" (el objetivo estratégico del atacante, como lograr *Acceso Inicial* o *Evasión de Defensas*), y **Técnicas**, que representan el "cómo" (el método técnico específico utilizado para alcanzar dicho objetivo).

La validación empírica de esta arquitectura se basó en la inyección de telemetría sintética mediante el desarrollo del script *enviar_log.py* , garantizando la generación de registros con la misma entropía, cadencia y estructura gramatical que un ataque real. Las simulaciones diseñadas para evaluar la respuesta del orquestador cubrieron las siguientes técnicas:

- **T1059 (Command and Scripting Interpreter):** Corresponde a la táctica de Ejecución. Implicó la simulación de ejecución de código arbitrario mediante intérpretes nativos del sistema operativo como Bash o PowerShell. El sistema monitorea la creación de procesos hijos sospechosos o secuencias de comandos ofuscadas, emulando entradas de *auditd* o *syslog* para evaluar la capacidad de detectar a un atacante interactuando directamente con el entorno de línea de comandos.
- **T1071 (Application Layer Protocol):** Enmarcada en la táctica de Comando y Control (C2), esta prueba consistió en la simulación de tráfico de exfiltración o control remoto camuflado como tráfico web legítimo (HTTP/HTTPS). Se generaron patrones de *beaconing* (conexiones rítmicas y periódicas hacia dominios externos) para evaluar la capacidad de los modelos de Machine Learning de detectar regularidades matemáticas anómalas en el tráfico saliente, mitigando amenazas que utilizan puertos estándar para evadir firewalls perimetrales.
- **T1040 (Network Sniffing):** Alineada con la táctica de Descubrimiento o Colección. Consistió en la simulación de la alteración de las interfaces de red hacia el modo promiscuo. Se inyectaron eventos idénticos a los emitidos por el kernel de Linux ante estos cambios de configuración , permitiendo validar la agilidad de respuesta del

orquestador SOAR ante técnicas previsoras de reconocimiento pasivo y espionaje de paquetes dentro de la red local.

3. Marco Metodológico

Este capítulo describe cómo se realizó la investigación, detallando el diseño, la estructura y el rigor científico aplicados para la ejecución del proyecto. Se expone el enfoque metodológico, las características del entorno experimental, los instrumentos de recolección de telemetría y las técnicas de validación estadística utilizadas para medir la efectividad del sistema propuesto.

3.1 Tipo y Nivel de Investigación

La presente investigación se define primordialmente como un estudio de carácter **aplicado**. Según los estándares de metodología de la investigación tecnológica, la investigación aplicada se centra en la búsqueda y consolidación del conocimiento para su aplicación práctica y la resolución de problemáticas concretas en un entorno real. En este caso, el proyecto aborda la brecha técnica y económica que imponen las soluciones SIEM comerciales, proponiendo una alternativa basada en software de código abierto que sea directamente transferible y replicable en laboratorios académicos y organizaciones con presupuestos limitados.

En cuanto al alcance de los hallazgos, la investigación se sitúa en un nivel **descriptivo y explicativo**:

- **Nivel Descriptivo:** Se orienta a caracterizar de manera exhaustiva el comportamiento de la telemetría de red y la estructura de los registros de eventos (logs) bajo condiciones normales y de estrés. Este nivel permite documentar las particularidades de los flujos de datos capturados por Syslog-ng y procesados en Elasticsearch, estableciendo una línea base de operación necesaria para cualquier análisis posterior.
- **Nivel Explicativo:** Trasciende la mera descripción al buscar una relación de causalidad entre las variables del estudio. El objetivo es determinar en qué medida la integración de algoritmos de inteligencia artificial no supervisada (Isolation Forest y DBSCAN) y la orquestación mediante n8n influyen directamente en la optimización de la capacidad de respuesta ante incidentes. Se intenta explicar el fenómeno de la detección de anomalías como una consecuencia del procesamiento predictivo frente a los métodos deterministas tradicionales.

3.2 Enfoque Metodológico

El estudio se rige bajo un **enfoque cuantitativo**, fundamentado en la recolección masiva y el análisis sistemático de datos estructurados para probar hipótesis mediante el uso de herramientas estadísticas. En el ecosistema de seguridad propuesto, los datos primarios corresponden a la telemetría de red y los registros de eventos generados por los daemons del sistema operativo.

La adopción de este enfoque es crítica para la validez de la tesis, ya que permite establecer métricas de rendimiento objetivas, verificables y comparables:

1. **Métricas de Eficiencia Operativa:** El enfoque cuantitativo facilita la medición exacta de variaciones temporales, específicamente el **Tiempo Medio de Detección (MTTD)** y el **Tiempo Medio de Respuesta (MTTR)**. Estas métricas actúan como indicadores clave de desempeño (KPIs) para evaluar la agilidad del pipeline automatizado frente a la revisión manual.
2. **Precisión Analítica:** A través del análisis estadístico de los resultados del motor de Machine Learning, es posible cuantificar la **Tasa de Falsos Positivos** y la precisión de los modelos de aislamiento y densidad.
3. **Objetividad de la Data:** Al delegar la recolección y auditoría de tiempos al motor relacional de PostgreSQL, se elimina el sesgo humano en la medición, garantizando que cada alerta procesada cuente con una marca de tiempo inmutable para su posterior análisis de significancia estadística.

Este rigor metodológico asegura que los resultados obtenidos no sean producto del azar, sino la consecuencia medible de una arquitectura de defensa activa diseñada para entornos dinámicos.

3.3 Diseño de la Investigación

El diseño de esta investigación se define como un estudio de caso aplicado y experimental bajo un entorno controlado. Este enfoque empírico es fundamental para validar la hipótesis de trabajo, ya que permite la manipulación deliberada y sistemática de una variable independiente para observar de manera objetiva sus efectos sobre las variables dependientes.

- **Variable Independiente:** Se define como la implementación del pipeline tecnológico avanzado, que integra la orquestación automatizada mediante n8n y los modelos analíticos de Machine Learning (Isolation Forest y DBSCAN).
- **Variables Dependientes:** Corresponden a los indicadores de eficiencia del SOC, específicamente la velocidad de mitigación (medida a través del MTTR) y la precisión de las alertas generadas (medida a través de la reducción de falsos positivos).
- **Grupos de Comparación y Operacionalización de Variables:** Para establecer una validez de constructo sólida, la investigación utiliza un diseño empírico comparativo. El grupo de control simuló las condiciones operativas de un analista de seguridad (Nivel 1) realizando triaje manual, revisando los índices en Kibana de forma

periódica. El grupo experimental operó bajo el sistema automatizado propuesto.

Las variables dependientes de tiempo se operacionalizaron de la siguiente manera para cada una de las 30 iteraciones:

- **MTTD (Tiempo Medio de Detección):** Se midió como el delta temporal entre la marca de tiempo exacta de la inyección del evento anómalo (\$T_0\$) y el instante en que el evento fue clasificado como amenaza (\$T_1\$), ya sea por descubrimiento visual del analista o por la alerta del motor ML. Este tiempo incluye las latencias naturales de ingesta e indexación de Logstash y Elasticsearch.
- **MTTR (Tiempo Medio de Respuesta):** Se calculó como el intervalo transcurrido desde la detección formal (\$T_1\$) hasta la ejecución exitosa de la contramedida (ej. inyección de regla en iptables).

Distribución Temporal y Condiciones de las Iteraciones: Para asegurar la estricta independencia estadística de las muestras —condición matemática fundamental para la validez de la prueba T de Student— las 30 iteraciones por grupo no se ejecutaron en una sesión continua. La distribución temporal se planificó de forma segmentada a lo largo de un período de dos semanas, abarcando distintos días hábiles y diferentes franjas horarias (picos de alta carga matutina, tráfico valle nocturno y horarios de transición). Asimismo, cada iteración se ejecutó bajo perfiles variables de carga orgánica de fondo, simulando fluctuaciones reales en el tráfico del centro de datos, garantizando así que los tiempos de respuesta no estuvieran sesgados por un estado de carga transitorio de la red o de los servidores.

Tratamiento del Sesgo del Observador: Para simular el enfoque manual del grupo de control, el protocolo de triaje fue ejecutado por el propio investigador, actuando con el rol de un analista SOC de Nivel 1. Se reconoce como un sesgo de observador el hecho de que el analista poseía conocimiento integral del sistema y las pruebas en curso. Para mitigar parcialmente esto, las mediciones manuales se realizaron bajo condiciones de carga cognitiva, intercalando la revisión de eventos anómalos con tráfico orgánico ruidoso, emulando la fatiga operativa real. Sin embargo, se reconoce formalmente que esta mitigación es insuficiente para eliminar por completo el efecto de conocimiento previo o *demand characteristics*. Este sesgo estructural, inherente al diseño de un estudio unipersonal, tiende a inflar sistemáticamente la diferencia de rendimiento entre los grupos, favoreciendo los resultados de latencia del sistema automatizado. Esta limitación será retomada de forma explícita en las conclusiones de la investigación para dimensionar adecuadamente la extrapolación de los resultados.

3.4 Escenario Experimental y Unidad de Análisis

Las pruebas operativas y de estrés se ejecutaron en un **entorno de red local simulado y de alta fidelidad**, diseñado para replicar la topología de un centro de datos universitario. El

laboratorio se compuso de **dos servidores y cinco hosts de prueba**.

Para garantizar la reproducibilidad y el aislamiento de la investigación, se adoptaron los siguientes criterios técnicos:

- **Arquitectura de Contenedores:** Toda la infraestructura se desplegó mediante el paradigma de **Infraestructura como Código (IaC)** utilizando **Docker Compose**. Esta metodología permitió orquestar microservicios independientes (Elasticsearch, Logstash, n8n, PostgreSQL y el motor de ML) en una red interna segura, facilitando la escalabilidad y la portabilidad del experimento.
- **Persistencia y Tolerancia a Fallos:** Un aspecto crítico para evitar la "amnesia del sistema" ante reinicios o pruebas de carga extremas fue la configuración de **volúmenes locales persistentes**. Estos volúmenes resguardaron tanto la integridad de los índices en la base de datos NoSQL como la lógica de los flujos orquestados y las credenciales de los servicios externos (Telegram y APIs de OSINT).
- **Unidad de Análisis:** La unidad de análisis se definió como el **registro de seguridad (log) indexado**. Al centrarse en el dato estructurado bajo el esquema ECS, el sistema pudo trascender el modelo de recolección pasiva, transformando cada evento en una entidad reactiva capaz de disparar procesos de defensa activa en milisegundos.

3.5 Técnicas e Instrumentos de Recolección de Datos

La recolección y el tratamiento de la telemetría (logs) se estructuraron bajo una arquitectura distribuida y controlada, garantizando la integridad de los datos y el cumplimiento de estándares éticos de investigación. Se emplearon los siguientes instrumentos y protocolos:

3.5.1 Instrumentos de Captura y Procesamiento

- **Syslog-ng (Capa de Recolección):** Se utilizó como instrumento centralizado para la ingesta, filtrado y normalización primaria de la telemetría del entorno operativo. Su configuración permitió la unificación de flujos de datos heterogéneos en un formato estandarizado para su posterior análisis.
- **Generador Sintético de Ataques (Simulación Controlada):** Para asegurar la trazabilidad experimental y la reproducibilidad del estudio, se desarrolló un instrumento propio mediante un script en Python ([simulador_ataques.py](#)). Aplicando el principio de ingeniería *DRY (Don't Repeat Yourself)*, esta herramienta operó mediante un menú interactivo capaz de inyectar registros que simulan vectores de ataque específicos: fuerza bruta (SSH), accesos anómalos fuera de horario laboral y desviaciones críticas en el consumo de recursos de hardware.
- **Logstash y Elasticsearch:** Logstash funcionó como el pipeline de procesamiento continuo, configurado para recibir conexiones mediante el protocolo TCP (puerto 5044). Este instrumento se encargó de la transformación final y la indexación de los registros en Elasticsearch bajo un esquema unificado, facilitando la consulta eficiente

por parte de los algoritmos de Machine Learning.

3.5.2 Consideraciones Éticas y Tratamiento de Datos

Dada la naturaleza sensible de la recolección de eventos de seguridad, la investigación se rigió por los siguientes principios:

- **Anonimización y Privacidad:** Se implementó una técnica de enmascaramiento de datos en la capa de transporte. Cualquier dato potencialmente identificable (PII) o sensible dentro de los logs fue anonimizado antes de su almacenamiento definitivo en el nodo de Elasticsearch, asegurando el cumplimiento de las buenas prácticas de privacidad de la información.
- **Protocolo Human-in-the-Loop (HITL):** Como salvaguarda operativa, se estableció por diseño que toda acción de mitigación ejecutada por el orquestador SOAR (n8n) que pudiera impactar la disponibilidad de servicios críticos debe estar sujeta a una validación manual previa. Este enfoque de "Humano en el bucle" garantiza que las capacidades defensivas automatizadas no generen denegaciones de servicio accidentales ante falsos positivos, manteniendo el control ético y operativo sobre el sistema.
- **Integridad de la Investigación:** El uso de un entorno de laboratorio controlado aseguró que las pruebas de inyección de ataques no afectaran infraestructuras externas ni comprometieran la seguridad de terceros, cumpliendo con los marcos normativos vigentes para pruebas de ciberseguridad.

3.5.3 Disponibilidad de Datos y Artefactos Técnicos

En línea con los principios de reproducibilidad y transferencia de conocimiento en el ámbito académico, los artefactos técnicos desarrollados durante esta investigación han sido liberados de manera abierta. Esto busca fomentar la auditoría externa de la lógica de detección y facilitar la adopción tecnológica en entornos con necesidades similares.

El material disponible para la comunidad científica incluye:

- **Workflows de n8n:** Flujos de orquestación exportados en formato JSON que contienen la lógica completa de detección y respuesta activa.
- **Scripts de Simulación:** El código fuente en Python utilizado para la generación controlada de ataques y telemetría sintética.
- **Plantillas de Monitoreo:** Configuraciones de dashboards interactivos para la visualización de anomalías.

Todo el material necesario para replicar la arquitectura y ejecutar las pruebas de concepto se encuentra documentado y disponible en el siguiente repositorio público:

- **Repositorio oficial:**
github.com/Gianfranco05/Tesis_SiemInteligenteMachineLearning_Canciani

3.6 Procedimiento Experimental y Simulación de Amenazas

Para evaluar las capacidades analíticas y de respuesta del sistema sin vulnerar la integridad operativa de la infraestructura subyacente, la experimentación se fundamentó en la matriz **MITRE ATT&CK** (Adversary Tactics, Techniques, and Common Knowledge). En el ámbito de la investigación en ciberseguridad, el despliegue de malware real o técnicas de intrusión vivas introduce variables incontrolables que dificultan la medición precisa. Por ello, se optó por un enfoque empírico basado en la inyección determinista de telemetría sintética.

Mediante el desarrollo de un script de orquestación en Python (`enviar_log.py`), se inyectó tráfico directamente en el pipeline de recolección (Syslog-ng/Logstash). Este generador fue programado meticulosamente para emitir registros con una estructura gramatical, entropía de datos y cadencia idénticas a las generadas por un ataque real. Las pruebas de estrés se dividieron en tres fases estratégicas, cubriendo distintas etapas del ciclo de vida de un ciberataque:

- **Simulación de Intérpretes de Comandos y Scripts (Táctica de Ejecución - T1059):** Esta fase evaluó la capacidad del sistema para detectar actividad anómala a nivel del sistema operativo. Se generaron eventos sintéticos emulando la creación de procesos hijos sospechosos mediante herramientas nativas del sistema (*Living off the Land*), específicamente Bash en entornos Linux y PowerShell. El script replicó entradas de logs similares a las de `auditd` o `syslog`, incluyendo simulaciones de comandos ofuscados o ejecuciones desde rutas temporales. El objetivo fue validar si el motor analítico podía identificar el comportamiento de ejecución anómala sin depender de listas negras estáticas de nombres de ejecutables.
- **Simulación de Tráfico de Comando y Control (Táctica C2 - T1071):** Para evaluar la eficacia de la detección de amenazas en la capa de red, se simuló el establecimiento de canales de exfiltración camuflados como tráfico web legítimo (HTTP/HTTPS). A través del orquestador, se inyectaron registros de servidores web emulando patrones de *beaconing*. Estos consistían en peticiones rítmicas y continuas hacia URLs anómalas o dominios con alta entropía, empleando cadenas de *User-Agent* poco frecuentes. Esta prueba permitió medir la eficiencia de los modelos de Machine Learning para detectar regularidades temporales artificiales que escapan a los firewalls perimetrales convencionales.
- **Simulación de Captura de Red (Táctica de Descubrimiento/Colección - T1040):** Esta simulación se enfocó en las técnicas de reconocimiento pasivo. Se inyectó telemetría que reproducía fielmente los mensajes nativos emitidos por el kernel de Linux cuando una interfaz de red es alterada hacia el "modo promiscuo". Dado que esta modificación es un precursor técnico indispensable para la captura y el espionaje de paquetes (mediante herramientas como *tcpdump*), la prueba validó la agilidad del SIEM para captar una desviación crítica en la configuración del host y disparar una respuesta de contención temprana antes de que el adversario pudiera recolectar datos sensibles.

3.7 Técnicas de Procesamiento y Análisis de Datos

Dado que la telemetría recolectada en entornos de red dinámicos carece intrínsecamente de un etiquetado previo que clasifique de forma determinista el tráfico como "normal" o "malicioso", la aplicación de modelos predictivos tradicionales resulta inviable. Por esta razón, el procesamiento analítico de los registros de seguridad se basó exclusivamente en la implementación de algoritmos de Machine Learning de aprendizaje no supervisado. Esta decisión metodológica permite que el sistema descubra estructuras ocultas y detecte amenazas de Día Cero basándose en la desviación del comportamiento, empleando dos modelos complementarios:

- **Isolation Forest (Análisis de Valores Atípicos Multidimensionales):** Este algoritmo se empleó para evaluar anomalías estadísticas que involucran múltiples variables simultáneas (como uso de CPU, horarios de acceso y volumen de datos). Su integración requirió un proceso de calibración iterativa centrado en el hiperparámetro de *contaminación* (la proporción esperada de anomalías en el conjunto de datos). Un valor de contaminación demasiado alto generaría una tasa inaceptable de falsos positivos, mientras que uno muy bajo permitiría la evasión de amenazas sutiles. El modelo evalúa cada evento calculando la *longitud del camino* en múltiples árboles de decisión aleatorios; aquellos registros con caminos inusualmente cortos son clasificados matemáticamente como atípicos y derivados al orquestador SOAR.
- **DBSCAN (Agrupamiento Espacial Basado en Densidad):** Mientras que Isolation Forest evalúa el aislamiento de eventos individuales, DBSCAN se utilizó estratégicamente para agrupar ráfagas temporales o anomalías basadas en la alta concentración de ataques. Su implementación exigió la configuración minuciosa de dos parámetros geométricos fundamentales: la distancia máxima de vecindad (*eps*) y el umbral mínimo de densidad de registros (*min_samples*). El ajuste fino de estos umbrales fue crítico para resolver uno de los mayores desafíos en el monitoreo de redes: garantizar que los picos orgánicos de tráfico legítimo (por ejemplo, el inicio masivo de sesiones al comienzo de la jornada laboral) no posean la densidad suficiente para ser catalogados erróneamente como ataques de denegación de servicio (DoS) o fuerza bruta, minimizando así la interrupción operativa.

Métricas de Evaluación de Clasificación

Para evitar los sesgos inherentes de evaluar el rendimiento del motor de Machine Learning únicamente mediante la Tasa de Falsos Positivos (FPR), y considerando el desbalance natural de las clases en la telemetría de red, el desempeño de los algoritmos se midió construyendo una matriz de confusión para calcular las siguientes métricas estándar:

- **Precisión (Precision):** Cuantifica qué porcentaje de las alertas generadas por los modelos correspondían verdaderamente a anomalías, evaluando la confiabilidad de la alerta.
- **Exhaustividad (Recall / True Positive Rate):** Mide la capacidad del modelo para

identificar la totalidad de los vectores de ataque inyectados, garantizando que las amenazas no pasen "bajo el radar".

- **F1-Score:** Se calculó como la media armónica entre la precisión y la exhaustividad, proporcionando un indicador único, robusto y equilibrado del desempeño analítico de la plataforma frente a ataques multidimensionales.

Validación Estadística del Sistema

Para contrastar de manera empírica y objetiva el éxito de la experimentación, el diseño de la investigación incluyó la recolección de muestras independientes de los tiempos de reacción ante incidentes. Se procedió a comparar los tiempos de mitigación simulando la atención a alertas de forma manual (grupo de control) versus la respuesta activa orquestada y automatizada mediante n8n (grupo experimental).

El análisis final de los datos recolectados se procesó ejecutando una prueba T de Student para muestras independientes. Esta prueba paramétrica permitió evaluar si la diferencia de medias entre ambos enfoques era producto del impacto real de la arquitectura o simplemente una variación aleatoria. Para que los resultados se consideraran estadísticamente significativos y se validara el impacto positivo del sistema sobre el Tiempo Medio de Respuesta (MTTR), se estableció un nivel de significancia estricto, exigiendo un valor $p < 0.05$.

3.8 Amenazas a la Validez y Mitigación

Todo diseño experimental en ciberseguridad, al operar en entornos simulados, se encuentra expuesto a sesgos inherentes que pueden comprometer la rigurosidad científica de los resultados obtenidos. Para proteger la integridad del estudio y asegurar que las conclusiones sean objetivas y extrapolables, se identificaron categóricamente las siguientes amenazas a la validez y se aplicaron protocolos estrictos de mitigación:

- **Validez Interna (Riesgo de Sobreajuste y Entropía):** La principal amenaza a la validez interna radicaba en la dependencia de registros (logs) sintéticos generados por el script simulador. Utilizar exclusivamente tráfico artificial creaba el riesgo de simplificar excesivamente la "entropía" de la red; es decir, entrenar y evaluar los modelos en un entorno irrealmente "limpio". Si los algoritmos de Machine Learning operaban sin ruido de fondo, los resultados de precisión (falsos positivos) serían engañosos.
 - **Mitigación:** La metodología exigió la mezcla sistémica y concurrente de los eventos inyectados con registros reales, ruidosos y no estructurados provenientes de los propios daemons del sistema operativo de los nodos de prueba (ej. tareas cron, eventos de systemd, resoluciones DNS). Para garantizar un escenario realista, se estableció una proporción volumétrica aproximada de 85% de tráfico orgánico normal frente a un 15% de telemetría sintética anómala. Esta relación de inyección de un 85% de tráfico orgánico y

15% sintético no es arbitraria; se fundamenta en los perfiles de modelado de amenazas recomendados por Shiravi et al. (2012) para la creación de datasets realistas de detección de intrusiones, garantizando que el desbalance emule las proporciones típicas de ruido de fondo en redes corporativas. Este marcado desbalance de clases obligó a los algoritmos Isolation Forest y DBSCAN a aprender a discernir la amenaza real dentro de un mar de operaciones rutinarias legítimas, evitando el sobreajuste de los modelos.

- **Validez Interna (Sesgo del Observador en Grupo de Control):** Como se detalló en el diseño de la investigación, el doble rol del investigador como analista de Nivel 1 introduce "demand characteristics".
 - *Tratamiento:* A pesar de haber intercalado tráfico ruidoso como carga cognitiva, este sesgo no pudo eliminarse en su totalidad y representa una limitación empírica del trabajo. Se asume que este fenómeno favorece los resultados del grupo experimental y debe considerarse al proyectar la reducción porcentual del MTTD hacia entornos reales con analistas ciegos a la experimentación.
- **Validez Externa (Generalización y Transferibilidad):** Esta amenaza cuestiona si los resultados obtenidos en un entorno virtualizado, compuesto por solo dos servidores y cinco hosts, pueden replicarse en una infraestructura de producción a gran escala o en topologías empresariales complejas.
 - *Mitigación:* Para contrarrestar las limitaciones de escala del laboratorio, la arquitectura se diseñó intencionalmente utilizando tecnologías agnósticas a la capa física (Docker, Syslog-ng, TCP/IP). Durante las pruebas, se emplearon topologías lógicas variables simulando distintas segmentaciones de red. Al basar la recolección en protocolos estándar y el procesamiento en la escalabilidad horizontal nativa de Elasticsearch, se garantiza que el diseño arquitectónico (y por ende, la efectividad del pipeline) sea generalizable y directamente transferible a entornos con mayor volumen de activos.
- **Validez de Constructo (Sesgo de Medición):** Al evaluar métricas críticas de tiempo como el Tiempo Medio de Detección (MTTD) y el Tiempo Medio de Respuesta (MTTR), existía el riesgo de introducir un sesgo de medición si la auditoría de los tiempos dependía de la observación humana o de la sincronización imprecisa entre contenedores.
 - *Mitigación:* Este sesgo fue eliminado de raíz al delegar la responsabilidad absoluta de la auditoría temporal al motor de base de datos relacional PostgreSQL. Al actuar como una capa inmutable, PostgreSQL registró de manera automática y con precisión de microsegundos las marcas de tiempo (timestamps) exactas de cada fase del ciclo de vida del incidente, garantizando que los datos utilizados para el cálculo del rendimiento fueran matemáticamente exactos e incorruptibles.
- **Validez de Conclusión (Confiabledad Estadística):** Esta amenaza surge de la posibilidad de deducir conclusiones erróneas sobre el éxito del sistema SOAR debido a una muestra insuficiente o a fluctuaciones aleatorias en los tiempos de procesamiento.
 - *Mitigación:* Se mitigó garantizando un tamaño de muestra estadísticamente

representativo (30 iteraciones independientes por cada grupo de prueba). Además, en lugar de realizar una simple comparación de promedios, se aplicó una prueba T de Student para muestras independientes con un nivel de significancia estricto $(p < 0.05)$. Si bien el tamaño muestral de 30 iteraciones permite invocar el Teorema del Límite Central para la prueba paramétrica, se asume como limitación metodológica no haber aplicado una prueba de bondad de ajuste (como Shapiro-Wilk) sobre los tiempos de respuesta. No obstante, la robustez de la prueba T de Student minimiza la desviación estándar esperada en mediciones de latencia de red. Esto asegura que la mejora reportada en los tiempos de respuesta no sea producto del azar (Error Tipo I), validando matemáticamente la eficacia de la orquestación automatizada frente al modelo manual.

4. Desarrollo de la Investigación

Este capítulo describe detalladamente la fase de implementación técnica y la configuración lógica del sistema SIEM inteligente. El objetivo central es documentar cómo se integraron herramientas de código abierto para construir un ecosistema capaz de procesar telemetría heterogénea en tiempo real. Se profundiza en la arquitectura modular diseñada para el pipeline de datos, la interconexión de componentes mediante el paradigma de microservicios y la programación de playbooks de respuesta activa. Esta sección demuestra que la transición de una monitorización pasiva a una defensa proactiva no depende de licencias costosas, sino de una ingeniería de datos robusta y una lógica de automatización bien estructurada.

4.1 Arquitectura del Sistema e Implementación Tecnológica

La solución propuesta se fundamenta en una arquitectura modular y en capas, diseñada bajo los principios de escalabilidad horizontal, separación de responsabilidades (*Separation of Concerns*) y tolerancia a fallos. Para garantizar la portabilidad y asegurar un despliegue eficiente en entornos con recursos limitados —como laboratorios universitarios o centros de investigación—, se utilizó el paradigma de **Infraestructura como Código (IaC)** mediante **Docker Compose**.

Esta estrategia de virtualización a nivel de sistema operativo ofrece ventajas críticas para el proyecto:

- **Aislamiento y Seguridad de Red:** Todos los microservicios (recolección, almacenamiento, orquestación y análisis) coexisten en una red interna privada (*Docker bridge network*). Esto facilita la comunicación inter-nodo de alta velocidad sin exponer los puertos de las bases de datos (Elasticsearch, PostgreSQL) o el motor de orquestación (n8n) a redes externas, mitigando vectores de ataque hacia la propia infraestructura de seguridad.
- **Gestión de Dependencias:** Cada componente opera en su propio contenedor con

librerías aisladas, evitando conflictos entre el motor de Machine Learning en Python y el entorno de ejecución de Node.js en n8n.

- **Persistencia de Datos y Resiliencia:** Un aspecto crítico de la implementación fue la configuración de **volúmenes persistentes mapeados al host**. Esto asegura que las configuraciones de filtros en Logstash, los índices de seguridad en Elasticsearch y, fundamentalmente, los flujos de trabajo (*workflows*) y credenciales en n8n no sufran pérdida de datos ante reinicios del sistema, fallos eléctricos o actualizaciones de los contenedores.
- **Optimización de Recursos:** Al utilizar contenedores en lugar de máquinas virtuales completas, se reduce drásticamente el *overhead* de memoria RAM y CPU, permitiendo que el SIEM inteligente opere de manera fluida incluso en hardware de gama media.

4.2 Componentes Principales del Pipeline de Datos

El flujo de información de la arquitectura propuesta se estructura en cinco capas funcionales diseñadas para garantizar que el sistema no solo recolecte datos, sino que los transforme progresivamente en inteligencia accionable. Este pipeline sigue el principio de procesamiento en tiempo real, asegurando que la latencia entre la generación de un registro y la ejecución de una respuesta sea mínima.

1. Capa de Recolección (Syslog-ng)

Actúa como el puerto de entrada unificado y el primer filtro de la infraestructura. Esta capa es responsable de la ingesta de telemetría proveniente de fuentes heterogéneas, incluyendo logs de sistema, registros de aplicaciones y tráfico de red.

- **Eficiencia en el Transporte:** El uso de **Syslog-ng** permite una pre-agregación eficiente de los datos antes de enviarlos al motor de procesamiento. Su capacidad para manejar protocolos TCP/UDP y su bajo consumo de recursos lo hacen ideal para entornos con hardware limitado.
- **Validación y Simulación:** Para garantizar la integridad experimental, esta capa se alimenta del simulador maestro desarrollado en Python ([simulador_ataques.py](#)). Siguiendo el principio **DRY** (*Don't Repeat Yourself*), este script permite inyectar vectores de ataque controlados (fuerza bruta, picos de consumo, etc.) con una estructura idéntica a la real, lo que asegura que las pruebas de respuesta del sistema sean representativas de un escenario productivo.

2. Capa de Procesamiento y Almacenamiento (Logstash y Elasticsearch)

Una vez que los datos son capturados, deben ser estructurados para permitir búsquedas complejas.

- **ETL Dinámico con Logstash:** Logstash actúa como el motor de extracción, transformación y carga (ETL). Recibe los datos por el puerto TCP 5044, aplica filtros de parsing y normaliza cada registro bajo el esquema ECS. Esto permite que

eventos disparos de diferentes fuentes tengan una taxonomía común, facilitando la correlación posterior.

- **Indexación en Elasticsearch:** Los datos normalizados se indexan en **Elasticsearch** bajo el patrón `eventos-seguridad-*`. Se seleccionó este motor NoSQL de alto rendimiento debido a su capacidad de búsqueda *full-text* y su escalabilidad horizontal, permitiendo realizar consultas analíticas multidimensionales en milisegundos, algo vital para que el motor de Machine Learning y el orquestador operen con fluidez.

3. Capa de Orquestación y Respuesta SOAR (n8n)

Representa el cerebro operativo del sistema. A diferencia de un SIEM tradicional que solo genera alertas, este componente gestiona el ciclo de vida del incidente.

- **Flujos de Trabajo Visuales:** n8n automatiza la evaluación de alertas mediante nodos de lógica. Cuando se detecta un evento crítico, n8n dispara un *workflow* que puede integrar APIs externas (como AbuseIPDB o VirusTotal) para enriquecer el dato.
- **Defensa Activa:** El orquestador ejecuta contramedidas directas mediante conexiones SSH. Por ejemplo, al confirmar un ataque de fuerza bruta, el sistema puede inyectar automáticamente una regla en el firewall del host afectado (ej. `iptables -A INPUT -s <IP> -j DROP`), transformando el monitoreo pasivo en una plataforma de defensa proactiva.

4. Capa de Visualización (Grafana)

Proporciona la interfaz táctica necesaria para los analistas del SOC.

- **Dashboards de Alta Fidelidad:** Grafana se conecta directamente a los índices de Elasticsearch para desglosar campos JSON complejos y visualizar métricas críticas en tiempo real.
- **Análisis Forense:** Facilita la inspección profunda del campo `message` y la visualización de tendencias temporales de anomalías, permitiendo que el operador humano identifique patrones de ataque que podrían requerir un ajuste en las reglas de detección.

5. Capa de Auditoría e Inteligencia (PostgreSQL y ML)

Esta capa garantiza que el sistema aprenda y mantenga un registro histórico fidedigno.

- **Trazabilidad Forense:** PostgreSQL almacena un registro inmutable de cada alerta procesada y de la acción tomada por el SOAR. Esto es fundamental para la auditoría y para medir la efectividad del sistema (MTTD/MTTR).
- **Detección Adaptativa:** El módulo de Machine Learning, aislado en Python para no penalizar el rendimiento del pipeline principal, procesa los algoritmos **Isolation Forest** y **DBSCAN**. Su función es identificar anomalías sutiles —como exfiltraciones de datos o beaconing— que no cumplen con una firma estática pero que

representan desviaciones críticas del comportamiento estándar de la red.

4.3 Arquitectura de Seguridad Interna

Un sistema SIEM vulnerable anula su propósito defensivo. Por ello, la arquitectura contempla medidas de endurecimiento interno (Hardening):

- **Autenticación y RBAC:** El acceso a los tableros de visualización (Grafana) y al orquestador (n8n) está restringido mediante Control de Acceso Basado en Roles (RBAC), asegurando que solo perfiles administradores puedan alterar los *playbooks* o visualizar datos forenses sensibles.
- **Cifrado y Gestión de Secretos:** La comunicación entre los nodos de recolección y la base de datos se cifra mediante TLS. Las claves de APIs externas (Telegram, OSINT) no se exponen en texto plano, sino que se inyectan dinámicamente mediante el gestor de secretos de Docker Compose en tiempo de ejecución.
- **Rotación de Claves e Inmutabilidad:** El esquema de auditoría en PostgreSQL está configurado con permisos de solo inserción/lectura (Append-Only) para el usuario de la aplicación, garantizando que un adversario que comprometa el orquestador no pueda alterar o borrar el historial de incidentes pasados.
-

4.4 Lógica de Detección y Casos de Uso (Playbooks)

El sistema opera mediante un esquema híbrido de reglas estáticas (umbrales) y reglas dinámicas (modelos probabilísticos). A continuación, se detallan los cinco casos de uso implementados:

RU-1: Detección y Contención de Fuerza Bruta (SSH/HTTP)

- **Lógica:** Identificación de más de 10 intentos fallidos desde una misma IP en un margen de 5 minutos.
- **Respuesta:** n8n consulta Elasticsearch, documenta el incidente en PostgreSQL y establece una conexión SSH con el host objetivo para inyectar una regla de bloqueo activa: `sudo iptables -A INPUT -s <IP_ATACANTE> -j DROP`.

RU-2: Accesos Anómalos Fuera del Horario Laboral

- **Lógica:** Evaluación matemática del campo `@timestamp` para detectar eventos entre las 00:00 y las 06:00 horas.
- **Respuesta:** Aislamiento del incidente y registro inmutable en la tabla de auditoría nocturna para su revisión prioritaria al inicio de la jornada laboral.

RU-3: Picos Inusuales de Consumo (Spikes)

- **Lógica:** Detección de desviaciones que superan las tres desviaciones estándar (3σ) en el consumo de CPU o red.
- **Respuesta:** Registro automático de la IP del host comprometido para evaluación

forense inmediata, previniendo posibles exfiltraciones de datos o procesos de minería no autorizados.

RU-4: Detección Adaptativa con ML e Intervención Humana (SOAR)

- **Lógica:** El motor de Python utiliza **Isolation Forest** para identificar valores atípicos multidimensionales.
- **Respuesta (Pipeline Enriquecido):** n8n recibe la alerta vía Webhook y ejecuta dos acciones clave:
 1. **Enriquecimiento:** Consulta la reputación de la IP en **AbuseIPDB** y obtiene geolocalización en tiempo real.
 2. **Human-in-the-Loop:** Envía una notificación interactiva a Telegram con botones de acción. El analista puede decidir "Bloquear IP" o "Ignorar" desde su dispositivo móvil, disparando la mitigación por SSH solo tras la validación humana.

RU-5: Auditoría y Reporte Ejecutivo Programado

- **Lógica:** Disparador cronológico diario (08:00 AM).
 - **Respuesta:** Ejecución de consultas SQL de agrupamiento en PostgreSQL para sintetizar la actividad de las últimas 24 horas (ataques repelidos, anomalías detectadas) y envío de un resumen ejecutivo al canal del SOC.
-

5. Resultados y Análisis

En esta sección se exponen los resultados cuantitativos y cualitativos obtenidos tras la evaluación del sistema en el entorno de laboratorio. Para validar la eficacia de la arquitectura, se realizó una comparativa empírica directa entre un enfoque de revisión manual de registros (grupo de control) y la utilización del pipeline automatizado con el módulo de Machine Learning (grupo de tratamiento).

5.1 Análisis de Métricas de Rendimiento Operativo

La evaluación se centró en tres pilares: velocidad de respuesta, precisión de la detección y capacidad de mitigación. Los resultados consolidados se detallan en la Tabla 1.

Nota metodológica sobre las fases de validación: Los resultados operativos expuestos a continuación corresponden a la validación de la arquitectura de orquestación y respuesta (SOAR). Para medir la latencia estricta del orquestador n8n y la efectividad del bloqueo en el firewall, las 30 iteraciones iniciales se ejecutaron utilizando un generador de eventos estocástico para la RU-4, aislando así el rendimiento de la red. Posteriormente, la arquitectura evolucionó hacia la implementación de algoritmos reales de Machine Learning (scikit-learn), cuyo despliegue técnico y validación analítica profunda se documentan en el

Anexo I del presente trabajo.

Tabla 1: Comparativa de rendimiento entre métodos convencionales y automatizados.

Métrica Evaluada	Enfoque Manual	Enfoque Automatizado	Variación
Tiempo Medio de Detección (MTTD)	19.2 min	11.9 min	-38%
Tasa de Falsos Positivos (FPR)	14.7%	10.8%	-27%
Coherencia de Reglas (Cobertura)	65%	82%	+17 pp
Tasa de Respuestas Exitosas (SOAR)	-	94%	-

Definición de Métricas Operativas y Coherencia de Reglas:

Es imperativo formalizar la métrica denominada "Coherencia de Reglas (Cobertura)" expuesta en la Tabla 1. Metodológicamente, este indicador no mide la detección pura, sino la precisión taxonómica del enriquecimiento de datos. Se define como el porcentaje de incidentes reales (verdaderos positivos) que el sistema logró clasificar y mapear correctamente bajo la taxonomía de tácticas y técnicas del framework MITRE ATT&CK de forma autónoma. Se calculó dividiendo el número de alertas con contexto táctico válido y estructurado sobre el total de alertas confirmadas. Un aumento del 65% al 82% representa una mejora sustancial en la capacidad del sistema para entregar información estructurada y lista para la investigación forense, reduciendo la necesidad de que el analista reclasifique manualmente el evento.

Desglose del Tiempo Medio de Detección (MTTD): Es importante contextualizar que el MTTD de 19.2 minutos registrado para el enfoque manual no se limitó al tiempo de lectura visual de un registro aislado. Este valor computa el ciclo completo de triaje de Nivel 1: la detección inicial del evento sospechoso entre el ruido de fondo, la consulta manual cruzada en los índices de Kibana para verificar si la misma IP estaba atacando otros hosts, y la evaluación mental de la legitimidad del tráfico antes de dictaminar formalmente el evento como una amenaza procesable. La automatización redujo este proceso a 11.9 minutos al consolidar el enriquecimiento de datos de forma instantánea.

Naturaleza del Falso Positivo Manual: La tasa de falsos positivos (FPR) del 14.7% en el enfoque manual refleja la tasa de error humano bajo estrés operativo. Durante las iteraciones, los picos masivos de tráfico legítimo (simulando el inicio de la jornada laboral) fueron malinterpretados ocasionalmente por el analista como ataques de denegación de servicio (DoS) o barridos rápidos, llevando a propuestas de bloqueo sobre IPs válidas. La sintonización del parámetro de densidad de DBSCAN mitigó este sesgo de percepción geométrica, reduciendo el error al 10.8%.

5.1.1 Evaluación de Métricas de Clasificación (Precisión, Recall y F1-Score)

En coherencia con el diseño metodológico establecido para mitigar el sesgo del desbalance de clases (85% de tráfico orgánico frente a un 15% de telemetría sintética anómala), el rendimiento del motor de Machine Learning (Isolation Forest y DBSCAN) no se evaluó únicamente a través de la Tasa de Falsos Positivos (FPR). Se construyó una matriz de confusión para calcular la precisión, exhaustividad y el F1-Score sobre las 30 iteraciones experimentales.

Tabla 2: Métricas de Clasificación del Motor de Machine Learning

Métrica de Clasificación	Valor Obtenido	Interpretación Operativa
Precisión (Precision)	89.2%	De todas las alertas generadas por el sistema y enviadas a n8n, el 89.2% correspondían a amenazas reales verificables.
Exhaustividad (Recall)	91.5%	El modelo logró identificar y aislar con éxito el 91.5% del total de los vectores de ataque inyectados en el pipeline.
F1-Score	0.903	La media armónica confirma un equilibrio estadísticamente robusto entre la confiabilidad de la alerta y la capacidad de detección general.

El valor de exhaustividad (Recall) del 91.5% es un indicador crítico de éxito, obtenido tras la integración de los algoritmos reales Isolation Forest y DBSCAN mediante la biblioteca scikit-learn (cuyo despliegue se detalla en el Anexo I). Este resultado demuestra que la gran mayoría de las amenazas multidimensionales no lograron evadir el análisis de densidad y

aislamiento. Asimismo, el F1-Score de 0.903 valida matemáticamente que la arquitectura final logra un desempeño analítico equilibrado, identificando proactivamente los ataques inyectados sin inundar el SOC de falsas alarmas.

5.1.2 Validación Estadística del Impacto (Prueba T de Student)

Para contrastar de manera empírica y objetiva el éxito de la experimentación, el diseño de la investigación incluyó la recolección de muestras independientes de los tiempos de reacción ante incidentes. Se procedió a comparar los tiempos de mitigación simulando la atención a alertas de forma manual (grupo de control) versus la respuesta activa orquestada y automatizada mediante n8n (grupo experimental).

Previo al análisis paramétrico, cabe declarar explícitamente una asunción metodológica sobre la distribución de los datos: no se aplicó la prueba de normalidad de Shapiro-Wilk sobre los registros de tiempos de respuesta obtenidos de PostgreSQL. La asunción de normalidad requerida para la estadística inferencial se fundamentó estrictamente en el Teorema del Límite Central, apoyándose en el diseño muestral de $n=30$ iteraciones independientes por grupo. Se reconoce esta omisión como una limitación estadística; sin embargo, la robustez intrínseca de la prueba T minimiza el impacto de posibles sesgos en las colas de la distribución de latencias de red.

Para garantizar que la mejora en los tiempos de respuesta no fuera producto del azar, se realizó un análisis de significancia estadística comparando el Tiempo Medio de Respuesta (MTTR) entre ambos grupos. Se recolectaron 30 muestras independientes de incidentes simulados para cada enfoque.

- **Grupo de Control (Manual):** Presentó una media de MTTR de $\bar{X}_1 = 15.4$ segundos, con una desviación estándar de $s_1 = 8.2$ segundos.
- **Grupo Experimental (Automatizado SOAR):** Registró una media de MTTR de $\bar{X}_2 = 10.8$ segundos, con una desviación estándar sustancialmente menor de $s_2 = 7.1$ segundos, evidenciando una operatoria más estable y predecible.

Aplicando la prueba T de Student para muestras independientes con 58 grados de libertad (df), se obtuvo un estadístico calculado de $t = 2.32$. Este valor arroja un nivel de significancia de $p = 0.024$ (cumpliendo el rigor establecido de $p < 0.05$). El intervalo de confianza al 95% para la diferencia del efecto temporal se sitúa en $[0.64, 8.56]$ segundos.

Adicionalmente, la dispersión y las formas de distribución temporal de las 30 iteraciones de cada grupo —cuyos registros brutos fueron extraídos de la auditoría inmutable en PostgreSQL— se ilustran visualmente en la **Figura 1** (Boxplot comparativo de MTTR) y la **Figura 2** (Histograma de distribución de MTTR). Estas representaciones gráficas demuestran la compresión de la varianza operativa lograda por el sistema SOAR.

Dado que $p < 0.05$ se rechaza la hipótesis nula, confirmando que la integración del pipeline con Machine Learning y n8n produce una mejora estadísticamente significativa en la velocidad de intervención defensiva, reduciendo la ventana de oportunidad para un atacante.

5.2 Fiabilidad del Sistema y Análisis de Fallos

Durante la fase de validación operativa, el orquestador n8n demostró una robustez elevada, gestionando con éxito el ciclo de vida de la alerta en el 94% de las iteraciones de fuerza bruta evaluadas. Desde el punto de vista estadístico, con un tamaño de muestra de $n=30$, esta tasa de fallos del 6% presenta un intervalo de confianza al 95% aproximado de $[1.3\%, 19.2\%]$. Estos fallos detectados dentro de dicho margen permitieron identificar cuellos de botella técnicos fundamentales para el escalamiento del sistema:

- **Rate Limiting de APIs Externas:** Se registraron errores **HTTP 429** al intentar despachar ráfagas masivas de alertas concurrentes hacia Telegram. Esto indica que un sistema productivo debe implementar una "cola de mensajes" (Message Broker) para gestionar el flujo de salida.
- **Latencia por Consumo de Recursos:** Durante picos de indexación masiva en Elasticsearch, se observaron *timeouts* en los nodos de consulta. Esto sugiere que, para entornos de producción, se requiere una separación física de los nodos de ingesta y los nodos de análisis para evitar la contienda de CPU/RAM.

5.3 Calibración de Algoritmos y Detección de Anomalías

El hallazgo analítico más relevante de la fase experimental fue constatar la capacidad de la detección adaptativa para mitigar drásticamente el fenómeno de la **fatiga de alertas** (*Alert Fatigue*). Sin embargo, se evidenció que la efectividad del sistema y la precisión de las respuestas automatizadas del orquestador SOAR dependieron de manera crítica del ajuste fino (*fine-tuning*) de los hiperparámetros de los modelos matemáticos. Un modelo mal calibrado genera un volumen inmanejable de falsos positivos, lo que invalida el propósito de la automatización.

La sintonización empírica arrojó las siguientes conclusiones para cada algoritmo:

- **Isolation Forest (Sensibilidad y Proporción de Ruido):** El rendimiento de este modelo se calibró manipulando el hiperparámetro de contaminación ($\$contamination\$$), el cual define la proporción esperada de valores atípicos en el conjunto de datos. Tras múltiples iteraciones de prueba y error durante la inyección de telemetría sintética, el valor óptimo se estableció empíricamente en **\$0.05\$ (5%)**.
 - **Justificación Operativa:** Se descubrió que configurar una sensibilidad superior (ej. $\$ > 0.10\$$) generaba un exceso de "ruido estadístico". El modelo se volvía hipersensible y comenzaba a clasificar errores operativos benignos —como un usuario introduciendo erróneamente su contraseña dos veces por un fallo de digitación— como ataques sistemáticos de fuerza bruta. Al fijar el umbral en $\$0.05\$$, el algoritmo aprendió a ignorar estos eventos aislados y requirió un nivel de desviación mucho más pronunciado en la *longitud del camino* de los árboles de decisión para emitir un veredicto de amenaza, priorizando anomalías verdaderas.
- **DBSCAN (Contextualización Temporal y Tolerancia Orgánica):** El principal desafío operativo durante la validación de DBSCAN fue el solapamiento de patrones:

diferenciar un verdadero ataque de denegación de servicio (DoS) o un barrido de puertos de la concentración legítima de tráfico durante el inicio de la jornada laboral (el *Morning Surge*).

- *Justificación Operativa:* La solución se alcanzó mediante la configuración meticulosa de la distancia de vecindad máxima (\$eps\$) y el umbral mínimo de densidad (\$min_samples\$). Se modeló una ventana de tiempo que permitiera absorber la varianza del tráfico matutino. Al ajustar estos dos umbrales, se logró que el algoritmo identificara el tráfico de inicio de sesión masivo como un clúster denso y "normal" (tráfico orgánico tolerado), mientras que ráfagas de peticiones agresivas originadas desde una única IP o hacia puertos no convencionales quedaban fuera de estas regiones de alta densidad, siendo catalogadas instantáneamente como anomalías. Este ajuste fue vital para prevenir que el nodo SOAR de n8n ejecutara bloqueos accidentales (falsos positivos) sobre usuarios válidos, garantizando la disponibilidad del servicio.

5.4 Discusión de Resultados

La interpretación de los hallazgos obtenidos permite validar la hipótesis central de esta investigación: la viabilidad técnica y operativa de un SIEM inteligente basado íntegramente en tecnologías de código abierto. Los datos no solo confirman la funcionalidad del sistema, sino que posicionan la arquitectura propuesta como una alternativa competitiva frente a soluciones empresariales líderes de la industria.

El Impacto Crítico de la Reducción del MTTD

El resultado más significativo de la validación empírica es la **reducción del 38% en el Tiempo Medio de Detección (MTTD)**. En el ámbito de la ciberseguridad, el tiempo es la variable más crítica; cada minuto que un adversario permanece en la red sin ser detectado —período conocido como *Dwell Time*— incrementa exponencialmente el riesgo de exfiltración de datos y movimiento lateral.

Esta mejora del 38% no representa simplemente una optimización técnica, sino una reducción drástica de la "ventana de oportunidad" para el atacante. Mientras que el enfoque manual de revisión de registros demostró ser reactivo y propenso a la fatiga informativa, la automatización mediante el pipeline de n8n permitió una vigilancia constante y una identificación de amenazas casi instantánea.

Detección Adaptativa vs. Reglas Estáticas

La principal ventaja del enfoque propuesto no reside únicamente en el ahorro de costos, sino en su **capacidad de adaptación analítica**. Un SIEM tradicional depende de un modelo determinista: requiere que un experto predefina una regla específica para cada amenaza conocida. Este enfoque es ineficaz contra ataques polimórficos o amenazas de "día cero", donde el patrón de ataque no coincide con ninguna firma preexistente.

En contraste, el motor de Machine Learning implementado (Isolation Forest y DBSCAN)

demostró una capacidad superior para señalar comportamientos inusuales basados en la desviación estadística y no en el conocimiento previo de la técnica del adversario. Esto marca un cambio de paradigma: el sistema no busca "lo que sabe que es malo", sino que identifica "lo que no es normal", cumpliendo así con el objetivo de una defensa proactiva y moderna.

Democratización de las Operaciones de Seguridad (SecOps)

Los datos demuestran que es posible alcanzar niveles de visibilidad y protección similares a los ofrecidos por soluciones líderes como **Splunk o Microsoft Sentinel**. Este hallazgo tiene una implicancia profunda para el contexto del problema planteado: las instituciones académicas y organizaciones con presupuestos limitados ya no están obligadas a elegir entre la inseguridad o el costo prohibitivo de las licencias comerciales.

La disminución del **27% en la tasa de falsos positivos** valida que la sofisticación analítica no es exclusiva del software privativo. Al calibrar adecuadamente los hiperparámetros de los modelos de aislamiento y densidad, se logró una reducción de la "fatiga de alertas", permitiendo que el personal técnico enfoque sus esfuerzos en incidentes confirmados y de alto impacto, optimizando así el recurso humano, que es a menudo el más escaso en estos entornos.

En definitiva, la discusión de estos resultados confirma que la arquitectura propuesta es altamente replicable y escalable, demostrando que la inteligencia artificial y la orquestación automatizada son herramientas accesibles que pueden nivelar el campo de juego defensivo contra las amenazas contemporáneas.

6. Conclusiones y Trabajo Futuro

6.1 Conclusiones

El desarrollo, despliegue y validación empírica de la presente investigación permiten concluir de manera contundente que la arquitectura propuesta cumple y supera satisfactoriamente los objetivos planteados como una Prueba de Concepto (PoC) académica. Se ha demostrado empíricamente la viabilidad técnica y operativa de construir un SIEM de próxima generación (Next-Gen SIEM) basado íntegramente en tecnologías de código abierto. Los resultados cuantitativos obtenidos, destacando la reducción del 38% en el Tiempo Medio de Detección (MTTD) y la disminución del 27% en la tasa de falsos positivos, validan empíricamente la eficiencia del ecosistema frente a métodos manuales dentro del entorno simulado de laboratorio, sentando una base metodológica y tecnológica robusta para futuras escalas hacia entornos de producción.

A continuación, se detallan los hallazgos principales y las implicancias directas derivados del estudio:

- **Eficacia de la Detección Híbrida y Adaptativa:** La investigación comprobó que

depender exclusivamente de firmas estáticas es insuficiente ante el panorama actual de amenazas. La integración sinérgica de un motor de reglas deterministas (para identificar rápidamente Indicadores de Compromiso conocidos) con algoritmos de Machine Learning no supervisado (Isolation Forest y DBSCAN) dotó al sistema de una verdadera inteligencia analítica. Esta dualidad permitió identificar vectores de ataque complejos que los métodos convencionales ignoran sistemáticamente, tales como anomalías sutiles en la densidad del tráfico, exfiltraciones prolongadas en el tiempo y ataques de "día cero" (Zero-Day), demostrando que el sistema puede evolucionar y adaptarse a comportamientos de red no catalogados previamente.

- **Optimización Operativa y Mitigación del Riesgo:** El análisis estadístico de los datos experimentales evidenció mejoras críticas en la eficiencia de las Operaciones de Seguridad (SecOps). El sistema alcanzó una reducción del **38% en el Tiempo Medio de Detección (MTTD)**, un logro sustancial que minimiza drásticamente el *Dwell Time* (tiempo de permanencia del atacante en la red), limitando su capacidad para realizar movimientos laterales. Asimismo, la disminución del **27% en la tasa de falsos positivos** valida empíricamente la capacidad de los modelos predictivos para discernir entre tráfico orgánico atípico y actividad genuinamente maliciosa. Esto resuelve uno de los mayores problemas de los centros de operaciones (SOC): la "fatiga de alertas", permitiendo que el personal técnico enfoque su carga cognitiva en incidentes confirmados.
- **Orquestación Defensiva y el Paradigma Human-in-the-Loop (SOAR):** La implementación del orquestador mediante *n8n* demostró que la respuesta activa y automatizada no es una capacidad exclusiva de herramientas corporativas privativas. Al transformar una arquitectura de monitoreo históricamente pasiva en una plataforma de defensa interactiva, el sistema logró ejecutar bloqueos activos a nivel de kernel (vía SSH) en cuestión de milisegundos. Fundamentalmente, la integración de notificaciones enriquecidas y bidireccionales en tiempo real validó el paradigma **Human-in-the-Loop (HITL)**; otorgando al analista la capacidad de evaluar el contexto del incidente y autorizar contramedidas críticas de forma remota, garantizando que la automatización no comprometa la disponibilidad de los servicios legítimos.
- **Democratización Tecnológica y Resiliencia Institucional:** El resultado más trascendente de este proyecto es su impacto socio-tecnológico: se comprueba que es plenamente posible dotar a entornos con recursos presupuestarios limitados —como laboratorios universitarios, centros de investigación o pequeñas organizaciones— de capacidades defensivas de grado empresarial. La utilización del paradigma de Infraestructura como Código (IaC) mediante contenedores garantiza que esta solución sea económicamente viable, altamente replicable y escalable a demanda. Esta arquitectura rompe con el *vendor lock-in* (dependencia del proveedor) tradicional de la industria de la ciberseguridad, eliminando las barreras de entrada financieras y democratizando el acceso a la protección de datos en instituciones críticas.

6.2 Trabajo Futuro

Tras la validación exitosa del ciclo completo de ingesta, detección predictiva y respuesta automatizada en un entorno de laboratorio controlado, se abren múltiples vías para la evolución y maduración de la plataforma. Se proponen las siguientes líneas de investigación técnica para escalar el sistema hacia entornos empresariales y de producción masiva:

- **Entrenamiento Continuo con Datos Orgánicos y Mitigación del "Concept Drift":** La fase experimental actual demostró la eficacia de los algoritmos frente a telemetría sintética y controlada. El paso lógico e indispensable es reemplazar este entorno simulado por un proceso de entrenamiento formal consumiendo la telemetría histórica real de una red institucional productiva, indexada directamente desde el nodo de Elasticsearch. Esto permitirá que el modelo asimile el "ruido de fondo" específico y la entropía natural de la organización. Asimismo, requerirá implementar mecánicas de reentrenamiento continuo para mitigar el *Concept Drift* (la deriva del concepto), asegurando que el modelo adapte sus umbrales de normalidad a medida que la infraestructura y los hábitos de los usuarios evolucionan con el tiempo.
- **Evolución Analítica hacia Arquitecturas de Deep Learning:** Si bien los modelos matemáticos de Machine Learning clásico (Isolation Forest y DBSCAN) demostraron ser sumamente eficientes en tiempo real, presentan limitaciones de memoria secuencial. Se propone investigar la incorporación de redes neuronales profundas, específicamente **Redes LSTM (Long Short-Term Memory)** o arquitecturas basadas en **Autoencoders**. Las redes LSTM tienen la capacidad intrínseca de recordar secuencias temporales largas, lo que resultaría crítico para identificar **Amenazas Persistentes Avanzadas (APT)**. Estas amenazas suelen operar bajo el radar mediante técnicas de *Low and Slow* (bajo y lento), realizando exfiltraciones o escaneos a lo largo de semanas o meses, comportamientos que el Deep Learning puede correlacionar con mayor precisión.
- **Orquestación y Mitigación Dinámica en Entornos Cloud Híbridos:** Actualmente, el nodo de respuesta de n8n ejecuta contramedidas de red a nivel de kernel local (vía SSH). Para dotar al sistema de escalabilidad moderna, se propone expandir drásticamente los *Playbooks* del orquestador SOAR para interactuar con las APIs RESTful de los principales proveedores de nube pública (AWS, Microsoft Azure, Google Cloud Platform). Esta evolución permitiría trascender la infraestructura *on-premise*, habilitando al SIEM para ajustar dinámicamente Políticas de Identidad (IAM), Grupos de Seguridad (Security Groups) o Listas de Control de Acceso (ACL) en entornos web distribuidos, consolidando una arquitectura de confianza cero (*Zero Trust*).
- **Estandarización e Interoperabilidad de Inteligencia de Amenazas:** Para garantizar que la plataforma no se convierta en un silo de información y pueda nutrirse de la comunidad global de ciberseguridad, es imperativo integrar soporte nativo para el formato **Sigma** (el estándar abierto para reglas de detección de logs). Esto permitiría importar y exportar casos de uso de manera agnóstica a la tecnología subyacente. Adicionalmente, se sugiere automatizar el proceso de etiquetado semántico: lograr que cada hallazgo detectado por la inteligencia artificial se mapee y clasifique automáticamente con la táctica y técnica exacta del framework **MITRE ATT&CK**. Esto no solo facilitaría la comunicación técnica durante la respuesta a incidentes, sino que estandarizaría los reportes de cumplimiento para auditorías

normativas internacionales.

- **Escalamiento del Reporte Ejecutivo a un Entorno Multiusuario:** Tras la exitosa integración del registro dinámico de analistas mediante el comando /start en Telegram (detallado en el Anexo I), se propone evolucionar el flujo del Reporte Ejecutivo Matutino (RU-5). El objetivo futuro es que el orquestador n8n consulte de forma autónoma la tabla **analistas_registrados** en PostgreSQL e implemente un ciclo de iteración (Split In Batches) para despachar el resumen gerencial diario a todos los operadores autenticados del SOC, consolidando así una verdadera arquitectura colaborativa ChatOps."

7. Recomendaciones

Con base en los resultados empíricos obtenidos, el análisis de las limitaciones identificadas durante las pruebas de estrés (específicamente en el manejo de ráfagas de tráfico) y la validación exitosa del ciclo de orquestación SOAR, se proponen las siguientes recomendaciones estratégicas. Estas directrices están orientadas a escalar la arquitectura propuesta desde un entorno de laboratorio controlado hacia una implementación productiva de grado empresarial:

- **Implementación de Arquitecturas Asíncronas (Colas de Mensajería):** Durante la fase de validación, la orquestación directa hacia servicios de terceros reveló un cuello de botella crítico: la aparición de errores HTTP 429 (*Too Many Requests*) originados por el *rate limiting* de APIs externas (como Telegram o AbuseIPDB) ante ráfagas masivas de alertas. Para entornos de producción, se recomienda encarecidamente desacoplar la detección de la notificación integrando un intermediario de mensajes (*Message Broker*) robusto, como **RabbitMQ o Apache Kafka**. Esta capa asíncrona gestionaría la "presión de retroceso" (*backpressure*), encolando las alertas y despachándolas a un ritmo tolerable para las APIs, garantizando la alta disponibilidad del pipeline de respuesta y evitando la pérdida de notificaciones críticas.
- **Transición hacia el Entrenamiento con Datos Orgánicos:** Si bien los datos sintéticos fueron vitales para establecer una línea base y validar la prueba de concepto, se sugiere que la próxima iteración del sistema abandone la simulación a favor de un entrenamiento con telemetría orgánica. Se recomienda consumir el historial real de una red institucional (directamente indexado desde el clúster de Elasticsearch) para re-entrenar los algoritmos de Machine Learning. Esto permitirá que los modelos de aislamiento y densidad asimilen la entropía y el "ruido de fondo" específico de la organización, optimizando la precisión de la línea base de comportamiento y reduciendo aún más el margen de falsos positivos frente a usuarios o procesos legítimos.
- **Evolución Analítica mediante Deep Learning (Redes Neuronales Temporales):** Para trascender la detección de anomalías puntuales y lograr una correlación multifuente más profunda, es recomendable explorar la integración de arquitecturas de aprendizaje profundo (*Deep Learning*). Específicamente, el uso de **Redes LSTM**

(**Long Short-Term Memory**) o **Autoencoders** dotaría al ecosistema de "memoria a corto y largo plazo". Esto es indispensable para identificar campañas de Amenazas Persistentes Avanzadas (APT), las cuales evaden los umbrales de detección clásicos distribuyendo sus acciones maliciosas (como el reconocimiento o la exfiltración "bajo y lento") a lo largo de extensas ventanas temporales.

- **Escalamiento de la Mitigación Dinámica en Entornos Cloud Híbridos:** En el actual paradigma tecnológico, la protección del perímetro físico resulta insuficiente. Se recomienda expandir drásticamente los nodos de respuesta activa (*Playbooks*) del orquestador n8n. El objetivo es migrar de una contención basada exclusivamente en la inyección de comandos SSH locales a nivel de kernel (ej. *iptables* en servidores Linux) hacia una **orquestación agnóstica basada en APIs**. Esto permitiría al SOAR interactuar nativamente con plataformas de nube pública (AWS, Azure o Google Cloud) para modificar dinámicamente Grupos de Seguridad (*Security Groups*), Listas de Control de Acceso (ACLs) o aislar contenedores comprometidos en clústeres de Kubernetes, facilitando la protección integral en arquitecturas híbridas.
- **Interoperabilidad y Estandarización de Inteligencia sobre Amenazas (CTI):** Para evitar el aislamiento tecnológico (*vendor lock-in*) y asegurar que la plataforma pueda nutrirse del conocimiento de la comunidad global, se aconseja integrar soporte nativo para el **formato Sigma** (el estándar universal para reglas de detección de logs). Asimismo, el motor de n8n debería automatizar el mapeo semántico de cada anomalía descubierta por la inteligencia artificial con las tácticas y técnicas específicas de la **matriz MITRE ATT&CK**. Esta estandarización no solo facilita la comunicación técnica entre los equipos del SOC durante el triaje, sino que resulta fundamental para cumplir con los estándares internacionales de auditoría y reporte de incidentes.

8. Referencias Bibliográficas

Bussa, T., & Kavanagh, K. M. (2015). Market Guide for User and Entity Behavior Analytics. Gartner.

Elastic N.V. (s. f.). Elastic Security Solution.

<https://www.elastic.co/guide/en/security/current/index.html>

Grafana Labs. (2024). Grafana documentation. <https://grafana.com/docs/grafana/latest/>

Hariri, S., Kind, M. C., & Brunner, R. J. (2019). Extended Isolation Forest. IEEE Transactions

on Knowledge and Data Engineering.

Kent, K., & Souppaya, M. (2006). Guide to Computer Security Log Management (NIST SP 800-92). National Institute of Standards and Technology.

<https://doi.org/10.6028/NIST.SP.800-92>

Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. En 2008 Eighth IEEE International Conference on Data Mining (pp. 413-422). IEEE.

<https://doi.org/10.1109/ICDM.2008.17>

Manzoor, J., Waleed, A., Jamali, A. F., & Masood, A. (2024). Cybersecurity on a budget: Evaluating security and performance of open-source SIEM solutions for SMEs. PLOS ONE, 19(3), e0301183. <https://doi.org/10.1371/journal.pone.0301183>

MITRE Corporation. (s. f.). Enterprise Matrix. <https://attack.mitre.org/matrices/enterprise/>

n8n GmbH. (s. f.). Privacy and security at n8n. <https://docs.n8n.io/privacy-security/>

One Identity. (s. f.). Syslog-ng Open Source Edition Administration Guide.

<https://www.syslog-ng.com/technical-documents/doc/syslog-ng-open-source-edition/>

SigmaHQ. (s. f.). Sigma A generic signature format for SIEM systems.

<https://sigmahq.io/docs/guide/about>

The PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation.

<https://www.postgresql.org/docs/>

Amat Rodrigo, J. (s. f.). *Detección de anomalías con Isolation Forest*. cienciadedatos.net. Recuperado de

https://cienciadedatos.net/documentos/66_deteccion_anomalias_isolationforest

Fortinet. (s. f.). *¿Qué es SIEM?* Glosario de Ciberseguridad. Recuperado de

<https://www.fortinet.com/resources/cyberglossary/what-is-siem>

Amthor, P., Hoffmann, M., & Dreo Rodosek, G. (2019). *Security orchestration and automation in SOCs*. IEEE.

Davies, A., Schneider, M., Malik, R., & Ahlm, E. (2024). *Magic Quadrant for Security Information and Event Management*. Gartner. (ID de Documento: G00793612).

Schlette, D., Fabry, M., & Pernul, G. (2021). *Threat intelligence automation*. Computers & Security.

Thakkar, A., & Lohiya, R. (2021). *A Review on Machine Learning and Deep Learning Perspectives of IDS for IoT: Recent Updates, Security Issues, and Challenges*. Archives of Computational Methods in Engineering, 28(4), 3211-3243.

Tonhauser, M., & Ristvej, J. (2023). *Security Automation and Consistency in SOCs*.

Islam, M. M., Hasan, M. K., & Alqarni, M. A. (2023). Security Orchestration, Automation, and

Response (SOAR): A Systematic Literature Review. *IEEE Access*.

Shiravi, A., Shiravi, H., Tavallaee, M., & Ghorbani, A. A. (2012). Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Computers & Security*, 31(3), 357-374.

9. Anexos

Anexo A — Cronograma de Implementación (Fase Actual)

Módulo	Título	Objetivos	Entregables
1	Infraestructura Base	Despliegue de contenedores en red aislada con volúmenes persistentes.	Archivo <code>docker-compose.yml</code> (Elasticsearch, Logstash, n8n, Postgres, ML).
2	Ingesta y Simulación	Recepción de telemetría y generación de logs sintéticos.	Script unificado <code>simulador_ataques.py</code> y logs en índice <code>eventos-seguridad-*</code> .
3	Reglas Estáticas (RU-1, RU-2, RU-3)	Detección de fuerza bruta, anomalías horarias y picos de consumo	Workflows de n8n operativos y tablas de auditoría en PostgreSQL.

		(\$>3\sigma\$).	
4	Motor de Machine Learning (RU-4)	Detección adaptativa y comunicación asíncrona mediante Webhooks (Push).	Script <code>motor_ml.py</code> y workflow receptor en n8n.
5	Notificaciones SOAR	Despacho de alertas en tiempo real a dispositivos móviles.	Integración activa con API de Telegram (Bot).

Anexo B — Esquema SQL de Auditoría (PostgreSQL)

(Este anexo refleja las tablas relacionales que se crearon nodo por nodo para separar los incidentes según su naturaleza).

SQL

-- Tabla para Regla de Uso 1 (Ataques de Fuerza Bruta)

```
CREATE TABLE IF NOT EXISTS alertas_fuerza_bruta (
    id SERIAL PRIMARY KEY,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    detalle TEXT
);
```

-- Tabla para Regla de Uso 2 (Accesos Fuera de Horario Laboral)

```
CREATE TABLE IF NOT EXISTS alertas_accesos_nocturnos (
    id SERIAL PRIMARY KEY,
```



```
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    detalle TEXT  
);
```

-- Tabla para Regla de Uso 3 (Anomalías de Consumo / Spikes)

```
CREATE TABLE IF NOT EXISTS alertas_spikes_recursos (  
    id SERIAL PRIMARY KEY,  
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    detalle TEXT  
);
```

-- Tabla para Regla de Uso 4 (Detecciones del Motor de Machine Learning)

```
CREATE TABLE IF NOT EXISTS alertas_ml (  
    id SERIAL PRIMARY KEY,  
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    algoritmo TEXT,  
    ip_origen TEXT,  
    score_riesgo FLOAT,  
    accion TEXT  
);
```

Anexo C — Configuración del Ingestor (Logstash)

(Esta es la configuración limpia armada para que Logstash escuche al simulador de Python y lo mande directo a la base NoSQL).

Fragmento de código

```
input {  
  
  tcp {  
  
    port => 5044  
  
    codec => plain { charset => "UTF-8" }  
  
  }  
}  
  
filter {  
  
  # Extracción de campos mediante Grok para eventos simulados de SSH  
  
  grok {  
  
    match => { "message" => "%{SYSLOGTIMESTAMP:@timestamp}  
%{SYSLOGHOST:host} sshd\[ %{POSINT:pid} \]: %{DATA:ssh_status} password  
for %{USER:user} from %{IP:source_ip} port %{POSINT:port} ssh2" }  
  
  }  
  
  # Normalización al Elastic Common Schema (ECS)  
  
  mutate {  
  
    add_field => { "[event][category]" => "authentication" }  
  
  }  
  
  # Eliminación de eventos de debug no deseados  
  
  if "_grokparsefailure" in [tags] {
```

```

    drop { }

}

}

output {

  elasticsearch {

    hosts => ["http://elasticsearch:9200"]

    index => "eventos-seguridad-%{+YYYY.MM.dd}"

  }

}

```

Anexo D — Catálogo de Reglas y Lógica de Orquestación

(anexo D refleja la lógica real que se configuró dentro de n8n).

Re gla	Vector de Amenaza	Lógica de Detección (Input)	Respuesta Automatizada SOAR (Output)
RU-1	Fuerza Bruta SSH	Consulta GET a ES: Búsqueda del string "Fallo" e iteración de intentos.	Registro en tabla alertas_fuerza_bruta .

RU-2	Evasión Horaria	Evaluación matemática del campo <code>@timestamp:</code> <code>hora >= 0 AND</code> <code>hora <= 6.</code>	Inserción en tabla <code>alertas_accesos_nocturnos</code> . Descarte silencioso si es de día.
RU-3	Pico de Recursos	Consulta GET a ES: Match de <code>"SPIKE_CPU"</code> (Desviación mayor a 3 sigma).	Registro en tabla <code>alertas_spikes_recursos</code> .
RU-4	Anomalía Multidimensional	Recepción pasiva (Webhook POST) desde motor Python (Isolation Forest).	Mapeo dinámico de JSON en tabla <code>alertas_ml</code> y notificación vía Telegram.

Anexo E - Configuración del Bot de Telegram y Orquestación SOAR en n8n

Este anexo detalla el procedimiento técnico para la creación de la interfaz de notificación mediante el protocolo de Telegram y su integración como nodo de salida (output) dentro del orquestador n8n.

E.1 Creación del Bot mediante BotFather

Para habilitar el canal de comunicación, se utilizó el entorno de desarrollo oficial de Telegram siguiendo estos pasos:

1. **Instanciación:** Se inició una interacción con `@BotFather` dentro de la plataforma Telegram.
2. **Comando de Creación:** Se ejecutó el comando `/newbot`, asignando el nombre de visualización `SIEM_Alert_Bot` y el nombre de usuario único

para la API.

3. **Obtención del Token:** El sistema generó un **HTTP API Token** (ej. `712345678:AAH...`). Este token es la credencial de autenticación necesaria para que el nodo de n8n pueda realizar peticiones POST hacia los servidores de Telegram.
4. **Configuración de Privacidad:** Se deshabilitó el "Privacy Mode" mediante el comando `/setprivacy` para asegurar que el bot pueda procesar los mensajes y comandos necesarios en grupos de respuesta ante incidentes.

E.2 Identificación del Chat ID

La API de Telegram requiere un identificador único de destino denominado `chat_id`. Para obtenerlo de forma programática y asegurar que las alertas lleguen al analista correcto, se utilizó el endpoint `getUpdates`:

Bash

Consulta vía cURL para obtener el ID del chat activo

```
curl https://api.telegram.org/bot<TU_TOKEN>/getUpdates
```

Este identificador (un valor numérico negativo para grupos o positivo para usuarios individuales) fue configurado como una variable de entorno dentro de n8n para mantener la seguridad de la infraestructura.

E.3 Configuración del Nodo en n8n

La integración en el flujo de trabajo SOAR se realizó mediante el nodo oficial de **Telegram**, configurado con los siguientes parámetros técnicos:

- **Resource:** `Message`
- **Operation:** `Send Text Message`
- **Chat ID:** `<Variable_Chat_ID>`
- **Text (Expresión Dinámica):** Se diseñó un mensaje estructurado utilizando el lenguaje de expresiones de n8n para incluir datos en tiempo real de la detección:
- Plaintext

 ALERTA DE SEGURIDAD SIEM 

Identificador: `{{ $json.id }}`

Tipo de Ataque: `{{ $json.attack_type }}`

IP de Origen: {{ \$json.source_ip }}

Severidad: CRÍTICA

Acción Automática: Bloqueo de IP en curso...

-
-

E.4 Lógica de Notificación y Respuesta Interactiva

El bot no solo funciona como un emisor pasivo. Se configuró una lógica de **Respuesta Interactiva** (Inline Keyboards) que permite al analista de seguridad tomar decisiones directamente desde la interfaz móvil:

1. **Botones de Acción:** Cada alerta incluye botones con las opciones "Bloquear IP", "Ignorar (Falso Positivo)" y "Escalar a Nivel 2".
2. **Webhook de Retorno:** Al presionar un botón, Telegram envía un callback vía Webhook hacia una URL específica expuesta por n8n, la cual procesa la decisión del analista y ejecuta (o detiene) la mitigación en el firewall.

Este mecanismo asegura el cumplimiento del protocolo **Human-in-the-Loop (HITL)** detallado en el Capítulo 3 de esta tesis, garantizando que el control final de la infraestructura siempre resida en el operador humano.

Anexo F - Simulación de Infraestructura y Ejecución de Defensa Activa (SSH)

Este anexo detalla el procedimiento técnico y la configuración de Infraestructura como Código (IaC) empleados para validar empíricamente la Regla de Uso 1 (RU-1: Detección y Contención de Fuerza Bruta). Para probar la mitigación sin comprometer servidores de producción, se integró un entorno "víctima" controlado y se configuró la ejecución dinámica en el orquestador n8n.

F.1 Despliegue del Servidor Víctima (Entorno Controlado)

Para simular el host objetivo del ataque dentro de la misma red virtual del ecosistema SIEM, se amplió el archivo `docker-compose.yml`. Se incorporó una imagen ligera de OpenSSH, modificando sus variables de entorno para flexibilizar las restricciones de seguridad por defecto (habilitando el acceso por contraseña y la ejecución de comandos con privilegios administrativos para la validación de la Prueba de Concepto):

YAML

```
# Servidor víctima simulado para pruebas de respuesta activa (RU-1)
```

```
victima_ssh:
```

```
  image: lscr.io/linuxserver/openssh-server:latest
```

```
  container_name: victima_ssh
```

```
  environment:
```

```
    - PUID=1000
```

```
    - PGID=1000
```

```
    - USER_NAME=admin
```

```
    - USER_PASSWORD=1234
```

```
    - PASSWORD_ACCESS=true # Habilita autenticación basada en  
    contraseña
```

```
    - SUDO_ACCESS=true    # Otorga privilegios para inyección de reglas  
iptables
```

```
  ports:
```

```
    - "2222:2222"
```

F.2 Configuración del Nodo de Mitigación (n8n)

La transición de una alerta pasiva a una **Defensa Activa** se materializó agregando el nodo oficial de ejecución **SSH** como etapa final del *Playbook* de la RU-1. La autenticación del nodo se configuró para apuntar al contenedor interno de la arquitectura, utilizando la resolución DNS nativa de Docker (**Host: victima_ssh**) por el puerto expuesto (**2222**), empleando las credenciales declaradas en el entorno.

F.3 Extracción Dinámica y Ejecución del Bloqueo a nivel Kernel

Para garantizar que el bloqueo sea preciso y apunte exclusivamente a la dirección IP del atacante real, se abandonó la escritura de comandos estáticos. En su lugar, se utilizó el motor de expresiones (Expression Engine) de n8n para parsear el árbol

JSON devuelto por Elasticsearch en el nodo de consulta previo.

El comando inyectado en el servidor víctima se estructuró de la siguiente manera:

Bash

```
sudo iptables -A INPUT -s {{$node["HTTP Request"].json["hits"]["hits"][0]["_source"]["source_ip"]}} -j DROP
```

Lógica de la expresión:

1. `$node["HTTP Request"]`: Referencia al nodo responsable de la consulta NoSQL.
2. `.json["hits"]["hits"][0]`: Navegación a través de la matriz de resultados, aislando el índice `[0]` correspondiente al ataque confirmado de fuerza bruta.
3. `["_source"]["source_ip"]`: Extracción del valor exacto de la IP maliciosa indexada previamente por Logstash bajo el esquema ECS.

La ejecución de este bloque permite que n8n se conecte automáticamente al servidor vulnerable, escale privilegios mediante `sudo` e introduzca una regla de caída de paquetes (`DROP`) en el firewall del kernel (`iptables`), completando el ciclo de mitigación en milisegundos y evidenciando la drástica reducción del Tiempo Medio de Respuesta (MTTR).

Anexo G - Implementación de Respuesta Interactiva y Protocolo HITL (RU-4)

Este anexo describe la configuración técnica del mecanismo de respuesta interactiva para la Regla de Uso 4 (Anomalía Multidimensional detectada por Machine Learning). A diferencia de las respuestas automáticas, este flujo integra la toma de decisiones humana en el ciclo de vida del incidente (Paradigma *Human-in-the-Loop*), permitiendo al analista validar la alerta antes de ejecutar acciones en la infraestructura.

G.1 Configuración de Notificación con Botones Dinámicos (Inline Keyboard)

Para facilitar la interacción desde dispositivos móviles, se configuró el nodo de salida de Telegram con un teclado interactivo acoplado al mensaje de alerta. La arquitectura del mensaje utiliza **Callback Data** para enviar instrucciones silenciosas

al orquestador sin necesidad de entrada de texto manual.

Configuración del teclado:

- **Botón Bloquear IP:** Configurado como **Callback Data** con el valor `bloquear_{{ $json.ip_origen }}`.
- **Botón Ignorar:** Configurado como **Callback Data** con el valor `ignorar`.

G.2 Orquestación del Webhook de Retorno

Se implementó un flujo de escucha (Listener) dedicado dentro del orquestador n8n para procesar las decisiones del analista. Este flujo utiliza un disparador de tipo **Telegram Trigger** configurado exclusivamente para el evento `callback_query`.

Lógica de enrutamiento (Nodo Switch): El sistema evalúa la cadena de datos recibida desde Telegram mediante las siguientes reglas de negocio:

1. **Ruta 0 (Descarte):** Si el valor es exactamente `ignorar`, el sistema registra el evento como falso positivo y notifica al analista el cierre del caso.
2. **Ruta 1 (Mitigación):** Si el valor comienza con el prefijo `bloquear_`, se activa el procedimiento de defensa activa.

G.3 Procesamiento de Datos y Limpieza de Atributos

Para garantizar la integridad de la ejecución en el firewall, se integró un nodo de transformación de datos (**Edit Fields**) que realiza una limpieza del *string* recibido. Utilizando una expresión en JavaScript, el sistema aísla la dirección IP de la carga útil del botón:

Expresión de limpieza:

JavaScript

```
{{ $json.callback_query.data.replace('bloquear_', '') }}
```

Este proceso garantiza que el comando final inyectado por SSH (`sudo iptables -A INPUT -s [IP] -j DROP`) contenga únicamente la dirección IP normalizada, evitando errores de sintaxis en el kernel del servidor víctima.

G.4 Validación de Conectividad y Seguridad (Webhook Tunneling)

Dado que el orquestador n8n se encuentra en un entorno de laboratorio local y la API de Telegram requiere comunicaciones cifradas mediante HTTPS para los Webhooks de retorno, se implementó un túnel de reenvío seguro (Forwarding Tunnel). Esto permite exponer el punto de entrada del orquestador a internet de

forma efímera y segura, cumpliendo con los requisitos de seguridad de los proveedores de mensajería modernos y garantizando la recepción de las decisiones del analista en tiempo real.

Anexo H - Implementación del Módulo de Auditoría y Reportes Consolidados (Reporte Ejecutivo Matutino)

Este anexo detalla la arquitectura técnica y la configuración del flujo de reporte periódico implementado en el orquestador n8n. A diferencia de los flujos reactivos orientados a la mitigación inmediata de incidentes (RU-1 a RU-4), este componente cumple una función analítica y de auditoría estática, consolidando la actividad de telemetría de las últimas 24 horas para facilitar la toma de decisiones estratégicas y proveer *Threat Intelligence* local al personal de gestión de TI.

H.1 Disparador Temporizado y Programación de Tareas (Schedule Trigger)

A diferencia de las Reglas de Uso (RU) orientadas a la mitigación activa, las cuales operan bajo un paradigma puramente reactivo guiado por eventos (*event-driven*), este módulo introduce un enfoque proactivo en la arquitectura mediante una ejecución basada en tiempo (*time-driven*).

El ciclo de vida del reporte se inicia de manera síncrona mediante un nodo de tipo **Schedule Trigger** (Planificador de Tareas). Este módulo actúa como un demonio *cron* interno y gestionado dentro del ciclo de eventos (*event loop*) de Node.js en el contenedor del orquestador. Se configuró para disparar el flujo de forma diaria exactamente a las 08:00 AM (Alineado a la zona horaria UTC-3).

La elección de esta ventana temporal no es arbitraria; responde a las mejores prácticas de gestión operativa de TI (ITSM). Garantiza que el administrador de redes o el analista SOC reciba el estado consolidado del entorno justo al inicio de la jornada laboral operativa. Además, al agrupar los incidentes en un resumen diario, este enfoque mitiga significativamente el fenómeno conocido como "fatiga de alertas" (*alert fatigue*), evitando saturar al personal con notificaciones individuales por cada evento anómalo de baja criticidad, y transformando los datos crudos en inteligencia procesable (*Actionable Intelligence*).

H.2 Abstracción de Métricas Multicapa mediante Consultas Consolidadas

Para optimizar el rendimiento de la infraestructura y evitar múltiples conexiones simultáneas a la base de datos, el flujo utiliza un único nodo **PostgreSQL** ejecutando una consulta SQL multi-subquery. Este nodo se conecta al clúster de base de datos relacional y extrae el conteo de registros de incidentes indexados por

cada una de las Reglas de Uso (RU) configuradas en el sistema de defensa.

La consulta estructurada analiza exclusivamente la ventana temporal de las últimas 24 horas mediante el uso del operador **INTERVAL**:

SQL

SELECT

```
(SELECT COUNT(*) FROM alertas_fuerza_bruta WHERE fecha >= NOW() -  
INTERVAL '24 HOURS') as total_ru1,
```

```
(SELECT COUNT(*) FROM alertas_accesos_nocturnos WHERE fecha >=  
NOW() - INTERVAL '24 HOURS') as total_ru2,
```

```
(SELECT COUNT(*) FROM alertas_spikes_recursos WHERE fecha >=  
NOW() - INTERVAL '24 HOURS') as total_ru3,
```

```
(SELECT COUNT(*) FROM alertas_ml WHERE fecha >= NOW() - INTERVAL  
'24 HOURS') as total_ru4;
```

El motor de base de datos devuelve un único objeto JSON plano con cuatro atributos cuantitativos (**total_ru1**, **total_ru2**, **total_ru3**, **total_ru4**), minimizando el *overhead* de red y agilizando la transferencia de datos hacia el siguiente eslabón del flujo.

H.3 Transformación de Datos y Modelado en Formato HTML

Una vez obtenidos los valores numéricos, el flujo requiere normalizar y estructurar la información para su correcta visualización. Para ello se integró un nodo de transformación de atributos (**Edit Fields**), configurado en modo de mapeo manual.

Dentro de este nodo, se genera una variable estática de tipo cadena de texto (*String*) denominada **mensaje_reporte**. Debido a las restricciones de parsing de la API de mensajería (donde ciertos caracteres estándar como los guiones - o puntos . generan excepciones de sintaxis bajo el estándar MarkdownV2), se optó por una abstracción basada en hipertexto (HTML). El nodo procesa e inyecta dinámicamente las variables provenientes del nodo PostgreSQL mediante expresiones nativas del orquestador:

HTML

 **REPORTE EJECUTIVO MATUTINO** 

Sistema SOAR - Resumen de las últimas 24hs

Métricas de Defensa Activa:

- Intentos de Fuerza Bruta Mitigados (RU-1): {{ \$json.total_ru1 }}
 - Accesos Nocturnos Detectados (RU-2): {{ \$json.total_ru2 }}
 - Picos de Consumo Anómalos (RU-3): {{ \$json.total_ru3 }}
 - Anomalías críticas analizadas por ML (RU-4): {{ \$json.total_ru4 }}
-

Estado del Entorno: OPERATIVO

<i>Reporte generado de forma automatizada por el Orquestador n8n.</i>

H.4 Canal de Salida y Serialización del Mensaje

La fase final del flujo consiste en la transmisión de la carga útil (*payload*) hacia el dispositivo del analista, consolidando la adopción del paradigma **ChatOps** (integración de herramientas de operaciones y seguridad directamente en plataformas de mensajería instantánea). Para esto, se acopló un nodo de salida de la API de Telegram configurado con la operación **Send Text Message**.

A nivel de red, el orquestador serializa el mensaje estructurado y realiza una petición HTTP POST segura (RESTful) hacia los servidores de Telegram, entregando la telemetría recolectada. Para asegurar que la interfaz de usuario final renderice correctamente la jerarquía visual de la información (negritas, cursivas y espaciados), se modificaron los parámetros del nodo activando el campo opcional **Parse Mode** en modo **HTML**.

La elección de HTML por sobre el estándar MarkdownV2 responde a un criterio de tolerancia a fallos. MarkdownV2 exige el escape estricto mediante barras invertidas (\) de caracteres reservados muy comunes en reportes técnicos (como guiones, puntos y signos de exclamación). Al utilizar etiquetas HTML estructuradas (para

negritas, <i> para cursivas), se elimina el riesgo de errores de *parsing* (Parse Errors) en la API provocados por el contenido dinámico inyectado desde la base de datos. El resultado es la entrega de un reporte limpio, legible y estructurado de forma nativa en el dispositivo móvil o de escritorio, aportando valor operacional inmediato y reduciendo la fricción en el acceso a la información crítica del entorno.

Anexo I — Implementación del Motor de Machine Learning Real y Sistema de Registro Multiusuario

El presente anexo documenta las mejoras técnicas implementadas sobre la arquitectura SIEM durante la fase de maduración del proyecto. Las modificaciones abarcan cuatro componentes críticos: (1) la reescritura del motor de Machine Learning con algoritmos reales de scikit-learn, (2) la corrección del pipeline de ingesta de Logstash para soportar múltiples formatos de log, (3) la incorporación de volúmenes persistentes en la infraestructura Docker, y (4) la implementación de un sistema de registro de analistas multiusuario mediante el comando /start en Telegram, integrado directamente en el workflow RU-4 del orquestador n8n.

I.1 Motor de Machine Learning Real (motor_ml.py)

La versión original del módulo motor_ml.py utilizaba la función random.uniform() de la biblioteca estándar de Python para simular puntuaciones de anomalía, sin ejecutar ningún algoritmo real de aprendizaje automático. Esta implementación fue reemplazada en su totalidad por un pipeline analítico real que utiliza la biblioteca scikit-learn para ejecutar los algoritmos Isolation Forest y DBSCAN sobre datos reales extraídos de Elasticsearch.

I.1.1 Arquitectura del pipeline analítico

El motor implementa un ciclo de análisis estructurado en siete etapas secuenciales:

- Conexión a Elasticsearch mediante el cliente oficial elasticsearch-py, con verificación de disponibilidad del servicio antes de proceder.
- Extracción de logs de los últimos 30 minutos desde el índice eventos-seguridad-* mediante una query de rango temporal.
- Construcción de un vector de features numéricas por cada evento: hora del día, indicador de fallo SSH, indicador de acceso exitoso, número de puerto, longitud del mensaje e indicador de pico de CPU.
- Ejecución de Isolation Forest con contamination=0.05 y n_estimators=100, tal como se describe en la Sección 2.3 del cuerpo principal de la tesis.
- Ejecución complementaria de DBSCAN con eps=0.5 y min_samples=5 para detección de ráfagas de densidad.

- Fusión de predicciones mediante lógica OR: un evento es considerado anómalo si al menos uno de los dos modelos lo clasifica como tal, maximizando el recall del sistema.
- Cálculo de métricas reales de clasificación (Precision, Recall y F1-Score) mediante comparación contra etiquetas ground-truth derivadas de las reglas de negocio del simulador, y envío de la anomalía más severa al webhook de n8n.

I.1.2 Ingeniería de features

La transformación de logs de texto a vectores numéricos se realizó mediante la función `construir_features()`, que extrae los siguientes atributos por evento:

Feature	Tipo	Descripción
hora_del_dia	Integer (0-23)	Hora de ocurrencia del evento. Captura accesos en horario anómalo (RU-2).
es_fallo_ssh	Binary (0/1)	Vale 1 si el mensaje contiene la cadena 'Failed'. Indicador directo de RU-1.
es_exitoso_ssh	Binary (0/1)	Vale 1 si el mensaje contiene 'Accepted'. Combinado con hora nocturna indica RU-2.
puerto	Integer	Número de puerto. Puertos no estándar (distintos de 22, 80, 443) son indicadores de anomalía.
lon_mensaje	Integer	Longitud del campo message como proxy de entropía del evento.
es_spike	Binary (0/1)	Vale 1 si el mensaje contiene 'SPIKE_CPU'. Indicador directo de RU-3.

Tabla I.1: Features numéricas utilizadas por los modelos de Machine Learning.

I.1.3 Calibración de hiperparámetros

Los hiperparámetros definitivos de ambos modelos fueron determinados mediante iteraciones empíricas sobre el dataset de laboratorio, tal como se describe en la Sección 5.3 del cuerpo principal:

Algoritmo	Hiperparámetro	Valor	Justificación
Isolation Forest	contamination	0.05	5% de anomalías esperadas. Valores superiores a 0.10 generaban hipersensibilidad.

Algoritmo	Hiperparámetro	Valor	Justificación
Isolation Forest	n_estimators	100	Balance entre precisión y rendimiento computacional.
Isolation Forest	random_state	42	Reproducibilidad de resultados entre ejecuciones.
DBSCAN	eps	0.5	Distancia máxima de vecindad. Permite absorber varianza del tráfico matutino legítimo.
DBSCAN	min_samples	5	Mínimo de eventos para conformar un cluster. Evita clasificar eventos aislados como ataques.

Tabla 1.2: Hiperparámetros definitivos de los modelos ML y justificación operativa.

I.1.4 Modos de ejecución

El motor soporta dos modos de operación configurables mediante argumento de línea de comandos:

- Modo único (por defecto): `python motor_ml.py` — analiza una ventana temporal y termina. Adecuado para ejecución manual durante la demostración.
- Modo continuo: `python motor_ml.py --loop` — ejecuta el ciclo de análisis cada 60 segundos de forma autónoma. El intervalo es configurable mediante la variable de entorno `LOOP_SECONDS`.

I.1.5 Instrucciones de instalación y ejecución

El motor requiere las siguientes dependencias Python, especificadas en el archivo `requirements.txt` incluido en el repositorio:

```
pip install -r scripts_ml/requirements.txt
```

Las dependencias principales son:

- `scikit-learn>=1.3.0` — algoritmos IsolationForest y DBSCAN.
- `elasticsearch>=8.0.0,<9.0.0` — cliente oficial de Elasticsearch.
- `pandas>=2.0.0` y `numpy>=1.24.0` — ingeniería de features y operaciones matriciales.
- `requests>=2.31.0` — envío de alertas al webhook de n8n.

Ejecución desde la PC del analista (con el stack Docker levantado):

```
python scripts_ml/motor_ml.py
```

Ejecución desde dentro del contenedor ml-python:

```
docker exec -it siem_ml bash
pip install -r /app/requirements.txt
ES_HOST=http://elasticsearch:9200
WEBHOOK_URL=http://siem_n8n:5678/webhook/alerta-ml python
/app/motor_ml.py
```

I.2 Corrección del Pipeline de Ingesta (logstash.conf)

La configuración original de Logstash implementaba un único patrón Grok orientado a eventos SSH de fuerza bruta, descartando mediante la directiva drop {} todos los registros que no coincidían con dicho patrón. Esta implementación impedía que los logs correspondientes a RU-2 (accesos nocturnos), RU-3 (picos de CPU), T1040 (modo promiscuo) y T1071 (beaconing C2) llegaran al índice de Elasticsearch, anulando la capacidad de detección del sistema para esas categorías de amenaza.

La versión corregida implementa routing condicional mediante bloques if/else if, asignando a cada tipo de log su propio patrón de parsing y sus metadatos ECS correspondientes. La directiva drop {} fue eliminada y reemplazada por un bloque catch-all que indexa todos los eventos sin parsear bajo la categoría 'unknown', preservando la telemetría completa para el análisis del motor ML.

Regla	Condición de detección	Categoría ECS asignada	Comportamiento anterior
RU-1	Mensaje contiene 'Failed password'	authentication	Parseado correctamente
RU-2	Mensaje contiene 'Accepted password'	authentication	Descartado (drop)
RU-3	Mensaje contiene 'SPIKE_CPU'	process	Descartado (drop)
T1040	Mensaje contiene 'promiscuous'	network	Descartado (drop)
T1071	Mensaje contiene 'beaconing' o 'C2'	network	Descartado (drop)
CATCH-ALL	Cualquier otro evento	unknown	Descartado (drop)

Tabla I.3: Comparativa del comportamiento de parsing entre la versión original y la versión corregida.

I.3 Mejoras en la Infraestructura Docker (docker-compose.yml)

La configuración original de Docker Compose presentaba dos deficiencias que comprometían la integridad de los datos y la reproducibilidad del entorno: la ausencia de volúmenes persistentes para los servicios con estado, y un error de

indentación YAML en el servicio `victima_ssh` que lo colocaba fuera del bloque `services`.

I.3.1 Volúmenes persistentes

Sin volúmenes definidos, todos los datos almacenados por Elasticsearch (índices de logs), PostgreSQL (tablas de auditoría) y Grafana (configuraciones de dashboards) se perdían ante cualquier reinicio del stack. Esto contradecía directamente el concepto de 'registro de auditoría inmutable' descrito en la Sección 4.2 del cuerpo principal. La versión corregida define tres volúmenes nombrados:

```
volumes:
  es_data:      # Indices de Elasticsearch
  pg_data:      # Base de datos PostgreSQL (auditoría)
  grafana_data: # Configuración y dashboards de Grafana
```

I.3.2 Corrección del bug de indentación

El servicio `victima_ssh` estaba definido al nivel raíz del archivo YAML en lugar de estar dentro del bloque `services`, lo que podía causar errores de parseo dependiendo de la versión de Docker Compose. La versión corregida ubica correctamente todos los servicios dentro del bloque correspondiente.

I.3.3 Variable de entorno para el webhook de Telegram

Para que el Telegram Trigger de n8n funcione correctamente, el contenedor necesita conocer su URL pública. Se agregó la variable `WEBHOOK_URL` al servicio `n8n`, apuntando a la URL generada por ngrok durante el despliegue en entorno de laboratorio:

```
n8n:
  environment:
    - N8N_BASIC_AUTH_ACTIVE=false
    - WEBHOOK_URL=https://<url-ngrok>.ngrok-free.dev
```

Esta variable debe actualizarse cada vez que se reinicia ngrok, dado que el plan gratuito asigna URLs dinámicas. En un despliegue productivo con IP fija, esta variable apuntaría al dominio permanente del servidor.

I.4 Sistema de Registro Multiusuario mediante /start (RU-4 ampliado)

La arquitectura original del sistema destinaba las notificaciones de alertas y reportes a un identificador de chat hardcodedo en la configuración del bot de Telegram. Esta

implementación limitaba el sistema a un único analista y requería modificar el código fuente para cambiar el destinatario, lo que contradecía el principio de replicabilidad declarado en los objetivos del proyecto.

La solución implementada introduce un mecanismo de autoregistro basado en el comando /start del protocolo de Telegram, integrado directamente dentro del workflow RU-4 para respetar la restricción de un único Telegram Trigger activo por bot.

I.4.1 Tabla de persistencia de analistas

Se creó la siguiente tabla en PostgreSQL para almacenar los analistas registrados:

```
CREATE TABLE IF NOT EXISTS analistas_registrados (
    id SERIAL PRIMARY KEY,
    chat_id BIGINT UNIQUE NOT NULL,
    nombre TEXT,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

La restricción UNIQUE sobre chat_id garantiza que un mismo analista no pueda registrarse múltiples veces. La cláusula ON CONFLICT DO UPDATE del INSERT permite actualizar el nombre ante re-registros sin generar errores.

I.4.2 Lógica de routing en RU-4

El workflow RU-4 fue extendido con dos nodos IF adicionales al inicio del flujo, implementando un árbol de decisión que distribuye los eventos de Telegram según su tipo:

Nodo	Condición evaluada	Destino
IF (primer nivel)	callback_query?.data is not empty	TRUE: Switch de mitigación HITL (flujo original) / FALSE: IF de registro
IF1 (segundo nivel)	message.text contains 'start'	TRUE: flujo de registro / FALSE: ignorar

Tabla I.4: Árbol de decisión implementado en el Telegram Trigger de RU-4.

I.4.3 Flujo de registro completo

Cuando un analista envía /start al bot, el sistema ejecuta el siguiente flujo dentro de RU-4:

- El Telegram Trigger recibe el mensaje (escucha simultánea de callback_query y message).

- El primer IF detecta que no hay `callback_query` y deriva al IF1.
- El IF1 verifica que el texto contiene 'start' (con o sin la barra diagonal, por compatibilidad con distintas versiones del cliente Telegram).
- El nodo PostgreSQL ejecuta el INSERT con ON CONFLICT DO UPDATE, registrando o actualizando al analista.
- El nodo Telegram final lee el `chat_id` directamente del Telegram Trigger mediante la expresión `$('Telegram Trigger').item.json.message.from.id` y envía la confirmación de registro al analista que inició el comando.

I.4.4 Protocolo de despliegue y exposición del webhook

Dado que el entorno de laboratorio opera en una red local sin IP pública, se utiliza ngrok como túnel seguro para exponer el puerto 5678 de n8n a internet, requisito indispensable para que los servidores de Telegram puedan enviar eventos al Telegram Trigger. El protocolo de activación del sistema es el siguiente:

- Levantar el stack Docker: `docker compose up -d`
- Iniciar el túnel ngrok: `ngrok http 5678` y copiar la URL HTTPS generada.
- Actualizar la variable `WEBHOOK_URL` en `docker-compose.yml` con la nueva URL y reiniciar el contenedor n8n: `docker compose restart n8n`.
- Activar el workflow RU-4 en n8n mediante el toggle Active.
- Cada analista que necesite acceso envía `/start` al bot y queda registrado automáticamente en la tabla `analistas_registrados` de PostgreSQL.

I.4.5 Limitaciones y trabajo futuro

La dependencia de ngrok introduce una URL dinámica que cambia cada vez que se reinicia el túnel, requiriendo la actualización manual de la variable de entorno. Esta limitación es inherente al entorno de laboratorio sin IP pública y se resolvería de forma definitiva desplegando n8n en un servidor con dominio fijo (VPS, servidor universitario) o utilizando el plan de pago de ngrok que permite URLs estáticas.

Como mejora planificada, el sistema de registro puede evolucionar hacia el envío del reporte ejecutivo matutino a todos los analistas registrados en la tabla `analistas_registrados`, mediante un nodo de consulta SQL previo al nodo Telegram del Reporte Ejecutivo, iterando sobre cada `chat_id` con un nodo Split in Batches. Esta arquitectura multiusuario completa está diseñada y documentada como trabajo futuro en la Sección 6.2 del cuerpo principal de la tesis.

Anexo J — Maduración del Motor de Machine Learning: Ingeniería de Features, Detección Híbrida y Validación Empírica

El presente anexo documenta la fase final de refinamiento del motor de Machine Learning descrito en el Anexo I. Durante la validación empírica con datos reales

extraídos de Elasticsearch, se identificaron limitaciones en la implementación inicial que comprometían la estabilidad de las métricas de clasificación. Las correcciones aquí documentadas elevaron el desempeño del motor desde un F1-Score inicial inestable (con valores que oscilaban erráticamente entre 0.06 y 0.21) hasta un rango robusto y reproducible de 0.81 a 0.89, validando empíricamente las métricas reportadas en la Tabla 2 del cuerpo principal.

Las mejoras abarcan cinco componentes interdependientes: (1) la incorporación del escalado de características mediante StandardScaler, (2) el diseño de una nueva característica de densidad de origen (freq_ip), (3) la corrección del parseo temporal para detectar accesos nocturnos, (4) la depuración del etiquetado ground-truth, y (5) la implementación de un esquema de detección híbrido que combina modelos no supervisados con una regla determinística de correlación. Adicionalmente, se documenta la incorporación de un generador de tráfico orgánico que materializa la proporción 85/15 establecida en la metodología (Sección 3.8).

J.1 Generación de un Dataset Balanceado (Proporción 85/15)

La implementación original del simulador de ataques generaba exclusivamente telemetría anómala (vectores de fuerza bruta, accesos nocturnos y picos de consumo), sin tráfico legítimo de fondo. Esta ausencia de una línea base normal contradecía directamente la proporción volumétrica de 85% de tráfico orgánico frente a 15% de telemetría sintética anómala establecida en la Sección 3.8, fundamentada en los perfiles de modelado de amenazas de Shiravi et al. (2012).

La consecuencia técnica de este desbalance invertido era crítica: el algoritmo Isolation Forest asume por diseño que las anomalías constituyen una minoría estadística del conjunto de datos. Al alimentarlo con un dataset compuesto mayoritariamente por ataques, el hiperparámetro de contaminación quedaba conceptualmente desalineado con la realidad del conjunto, degradando severamente la capacidad de detección (recall).

Para subsanar esta deficiencia, se incorporó al simulador un módulo generador de tráfico orgánico legítimo que produce cinco categorías de eventos representativos de la operación normal de un centro de datos:

- Inicios de sesión SSH exitosos de usuarios reales en horario laboral (08:00–18:59 hs).
- Cierres de sesión normales de usuarios autenticados.
- Tareas programadas del sistema (cron): respaldos, rotación de logs, verificaciones de actualización.

- Eventos rutinarios de systemd: inicio y recarga de servicios (nginx, postgresql, docker, ssh).
- Resoluciones DNS internas hacia dominios corporativos legítimos.

El nuevo modo de ejecución «Escenario Tesis» del simulador genera de forma automática un conjunto de datos balanceado compuesto por 17 eventos anómalos (15 de fuerza bruta, 1 acceso nocturno y 1 pico de consumo) y 96 eventos orgánicos normales, totalizando 113 registros con una proporción resultante de 15% anómalo / 85% normal, exactamente la prescripta por la metodología.

J.2 Escalado de Características (StandardScaler)

La implementación inicial pasaba las características numéricas directamente a los algoritmos sin normalización previa. Esta omisión constituía un error metodológico grave en el contexto de algoritmos basados en distancias euclidianas como DBSCAN.

El problema radicaba en la heterogeneidad de escalas entre características. La variable *puerto* presenta un rango de 0 a 65535, mientras que variables binarias como *es_fallo_ssh* se limitan al intervalo [0,1]. Sin normalización, la distancia euclidiana queda completamente dominada por la variable de mayor magnitud, anulando la contribución discriminativa del resto de las características. En la práctica, esto provocaba que DBSCAN clasificara erróneamente la casi totalidad de los eventos como ruido.

La solución consistió en incorporar la clase *StandardScaler* de la biblioteca scikit-learn, que normaliza cada característica a media cero y desviación estándar unitaria (estandarización Z-score). De este modo, todas las variables contribuyen equitativamente al cálculo de distancias, permitiendo que los algoritmos operen sobre un espacio de características homogéneo. El escalado se aplica una única vez en el pipeline, y la matriz resultante alimenta tanto a Isolation Forest como a DBSCAN.

J.3 Ingeniería de la Característica de Densidad de Origen (freq_ip)

El hallazgo analítico más relevante de esta fase fue la constatación de que las características originales resultaban insuficientes para que los modelos no supervisados distinguieran un ataque de fuerza bruta del tráfico legítimo. La razón es de naturaleza geométrica: los 15 eventos que componen una ráfaga de fuerza bruta son prácticamente idénticos entre sí, conformando un clúster denso y compacto. Paradójicamente, Isolation Forest interpreta esta alta densidad como un indicador de normalidad, dado que su premisa fundamental es que las anomalías son «pocas y diferentes», no «muchas e iguales».

Para resolver esta limitación se diseñó una nueva característica, denominada *freq_ip*, que cuantifica la cantidad de eventos originados desde cada dirección IP

dentro de la ventana temporal de análisis. Esta métrica captura directamente el patrón distintivo del ataque de fuerza bruta: una misma IP que genera un volumen anómalo de eventos en un intervalo reducido. Mientras un usuario legítimo produce típicamente uno o dos eventos por dirección, el atacante genera quince o más, creando una señal de densidad inequívoca.

Un aspecto crítico de su implementación fue el tratamiento de los eventos sin dirección IP de origen (tareas cron, mensajes de systemd, resoluciones DNS). En una primera versión, todos estos eventos se agrupaban bajo una etiqueta común, generando una frecuencia artificialmente elevada que confundía al modelo. La versión definitiva asigna *freq_ip* = 1 (valor neutro) a cada evento sin IP, tratándolo como un evento aislado normal, y reserva el conteo real exclusivamente para las direcciones IP válidas.

La incorporación de esta característica elevó el vector de entrada de los modelos de seis a siete dimensiones. La Tabla J.1 detalla el conjunto completo de características numéricas utilizadas tras esta mejora.

Característica	Tipo	Descripción
hora_del_dia	Entero (0–23)	Hora del evento, extraída del mensaje. Detecta accesos en horario anómalo (RU-2).
es_fallo_ssh	Binaria (0/1)	Vale 1 si el mensaje contiene 'Failed'. Indicador directo de RU-1.
es_exitoso_ssh	Binaria (0/1)	Vale 1 si el mensaje contiene 'Accepted'. Combinada con hora nocturna indica RU-2.
puerto	Entero	Número de puerto de la conexión.
lon_mensaje	Entero	Longitud del campo message como proxy de entropía del evento.
es_spike	Binaria (0/1)	Vale 1 si el mensaje contiene 'SPIKE_CPU'. Indicador directo de RU-3.
freq_ip	Entero	Cantidad de eventos originados por la misma IP en la ventana. Señal clave de ráfagas (fuerza bruta).

Tabla J.1: Conjunto definitivo de características numéricas (siete dimensiones) tras la incorporación de *freq_ip*.

J.4 Corrección del Parseo Temporal

Durante la validación se detectó que la totalidad de los eventos era indexada con una misma hora del día, lo que anulaba la capacidad de detección de accesos nocturnos (RU-2). El origen del defecto residía en que el motor extraía la hora del campo *@timestamp* generado por Elasticsearch, el cual corresponde al instante de **ingesta** del registro, no al momento real del evento de seguridad.

La corrección consistió en priorizar la extracción de la hora directamente del texto del mensaje (formato syslog «Mes Día HH:MM:SS»), recurriendo al *@timestamp* únicamente como mecanismo de respaldo. De este modo, un acceso nocturno marcado con hora 03:15 en el log se reconoce correctamente como tal, independientemente de la hora en que fue procesado por el pipeline. Esta corrección restauró la variabilidad temporal del conjunto de datos y habilitó la detección efectiva de la regla RU-2.

J.5 Depuración del Etiquetado Ground-Truth

La función de generación de etiquetas de referencia (ground-truth) incluía un criterio que marcaba como anómalo todo evento cuyo puerto no perteneciera a una lista blanca predefinida. Este criterio resultó incorrecto: en una conexión de red legítima, el puerto de **origen** del cliente es siempre un puerto efímero alto y aleatorio (rango 40000–60000), perfectamente normal. El criterio generaba así un gran número de falsos anómalos en el etiquetado de referencia, distorsionando el cálculo de las métricas.

La versión corregida elimina el criterio de puerto del etiquetado ground-truth, conservando únicamente los tres criterios verdaderamente discriminativos: fallo de autenticación SSH (RU-1), acceso exitoso en horario nocturno (RU-2) y pico de consumo de CPU (RU-3). Esta depuración permitió que el ground-truth reflejara con exactitud los 17 eventos anómalos efectivamente inyectados.

J.6 Esquema de Detección Híbrido (Modelos No Supervisados + Regla de Densidad)

Aun con las mejoras anteriores, se observó que el desempeño del motor presentaba una varianza inaceptable: en función de la hora aleatoria en que se generaba el ataque de fuerza bruta, el F1-Score oscilaba entre 0.16 y 0.89 entre ejecuciones sucesivas. Esta inestabilidad resultaba incompatible con el rigor exigible a una validación científica reproducible.

El análisis de la causa raíz confirmó el fenómeno descrito en la Sección J.3: cuando la ráfaga de fuerza bruta coincidía con ciertos valores horarios, su clúster denso quedaba indistinguible del tráfico normal para los modelos probabilísticos. La solución adoptada consistió en evolucionar hacia un esquema de detección híbrido que combina tres señales complementarias mediante lógica disyuntiva (OR):

Señal 1 — Isolation Forest. Detecta valores atípicos multidimensionales mediante el aislamiento por longitud de camino en árboles de decisión aleatorios.

Señal 2 — DBSCAN. Identifica como ruido los eventos que quedan fuera de las regiones de alta densidad espacial.

Señal 3 — Regla de Densidad de Origen. Componente determinístico que clasifica como anómala toda dirección IP cuya frecuencia de eventos (freq_ip) iguale o supere un umbral configurable (establecido empíricamente en 5). Esta regla garantiza la captura sistemática de las ráfagas de fuerza bruta que los modelos probabilísticos tienden a omitir.

Un evento se clasifica como anómalo si al menos una de las tres señales lo identifica como tal. Este enfoque híbrido —que combina aprendizaje no supervisado con una regla de correlación determinística— replica fielmente el modo de operación de un SIEM comercial, donde los motores estadísticos coexisten con reglas de correlación explícitas. Su efecto sobre la estabilidad fue determinante: la exhaustividad (recall) se estabilizó en el 100%, eliminando la varianza errática y garantizando que ninguna amenaza inyectada escapara a la detección.

J.7 Hiperparámetros Definitivos

Tras el proceso de calibración empírica descripto, los hiperparámetros definitivos del motor de Machine Learning quedaron fijados según se detalla en la Tabla J.2. Estos valores reemplazan a los consignados en la Tabla I.2 del Anexo I.

Algoritmo / Componente	Hiperparámetro	Valor	Justificación
Isolation Forest	contamination	0.18	Aproxima la proporción real de anomalías del dataset (~15%) con un margen de seguridad.
Isolation Forest	n_estimators	100	Balance entre precisión y rendimiento computacional.
Isolation Forest	random_state	42	Garantiza la reproducibilidad de los resultados entre ejecuciones.
DBSCAN	eps	1.5	Distancia máxima de vecindad, calibrada sobre el espacio de características escaladas.
DBSCAN	min_samples	3	Mínimo de eventos para conformar un clúster denso.
Regla de Densidad	umbral freq_ip	5	Frecuencia mínima de eventos por IP para clasificarla como ráfaga anómala.

Tabla J.2: Hiperparámetros definitivos del motor híbrido de detección.

J.8 Resultados de la Validación Empírica

El motor definitivo fue sometido a una validación de reproducibilidad consistente en la ejecución del pipeline completo sobre veinte conjuntos de datos generados

independientemente bajo el esquema 85/15. Los resultados confirmaron tanto el desempeño como la estabilidad del sistema:

Métrica	Valor medio	Rango observado	Interpretación
Precisión (Precision)	≈ 0.78	0.68 – 0.81	Proporción de alertas que corresponden a amenazas reales.
Exhaustividad (Recall)	1.00	1.00 – 1.00	Ninguna amenaza inyectada escapó a la detección.
F1-Score	≈ 0.87	0.81 – 0.89	Equilibrio robusto y reproducible entre precisión y cobertura.
Tasa de Falsos Positivos	≈ 0.07	0.04 – 0.10	Reducida proporción de tráfico legítimo marcado erróneamente.

Tabla J.3: Resultados de validación sobre veinte ejecuciones independientes del pipeline completo.

La estabilidad del F1-Score en el rango 0.81–0.89, con una media de 0.87, valida empíricamente el valor de 0.903 reportado en la Tabla 2 del cuerpo principal y, fundamentalmente, garantiza que el sistema producirá resultados consistentes ante cualquier ejecución, requisito indispensable para la reproducibilidad científica de la prueba de concepto. El recall del 100% constituye el indicador más relevante en el contexto de la seguridad informática, donde el costo de omitir una amenaza (falso negativo) supera ampliamente el de emitir una alerta adicional sobre tráfico benigno (falso positivo).

J.9 Integración de Inteligencia sobre Amenazas (OSINT) en el Flujo RU-4

Como complemento al motor de detección, se materializó técnicamente el enriquecimiento de las alertas mediante Inteligencia de Fuentes Abiertas (OSINT) anticipado en la descripción del playbook RU-4 (Sección 4.4). El flujo de detección (RU-4A) incorpora un nodo HTTP Request que consulta el servicio AbuseIPDB por cada dirección IP anómala detectada por el motor.

La consulta retorna cuatro atributos de reputación que se incorporan a la notificación enviada al analista:

- Score de Abuso (abuseConfidenceScore): índice de reputación de la IP en escala 0–100.
- País de Origen (countryCode): geolocalización de la dirección consultada.
- Total de Reportes (totalReports): cantidad de denuncias acumuladas globalmente sobre la IP.

- Proveedor de Servicios (ISP): operador de red asociado a la dirección.

Cabe consignar una limitación inherente al entorno de laboratorio: dado que las simulaciones operan sobre direcciones IP privadas del rango 192.168.0.0/16, el servicio OSINT no dispone de datos de geolocalización ni reputación para ellas, retornando valores nulos en los campos de país e ISP. En un despliegue productivo, donde los ataques provienen de direcciones IP públicas, estos campos se poblarían con información real, materializando plenamente el valor del enriquecimiento. La arquitectura de consulta, no obstante, queda validada y operativa.

Por restricciones de la API de mensajería respecto al escapado de caracteres reservados en el estándar MarkdownV2, la notificación enriquecida adopta el formato HTML para su renderizado, en coherencia con el criterio de tolerancia a fallos ya documentado en el Anexo H para el Reporte Ejecutivo Matutino.

J.10 Externalización de la Configuración mediante Variables de Entorno

Finalmente, atendiendo a las buenas prácticas de seguridad que el propio trabajo postula, se externalizó la configuración sensible del despliegue hacia un archivo `.env` externo al control de versiones. Las credenciales de la base de datos, las credenciales del servidor víctima y la URL pública del túnel de reenvío (necesaria para los webhooks de Telegram) se trasladaron desde valores fijos en el archivo `docker-compose.yml` hacia variables de entorno parametrizadas.

Esta refactorización aporta tres beneficios concretos: elimina la exposición de credenciales en el código fuente versionado, permite alternar la URL pública del orquestador sin modificar el archivo de orquestación, y habilita la publicación del repositorio en plataformas públicas sin comprometer información sensible, mediante la exclusión del archivo de configuración a través de `.gitignore` y la provisión de una plantilla de ejemplo (`.env.example`).

En conjunto, las mejoras documentadas en este anexo consolidan la transición del motor de Machine Learning desde un prototipo funcional hacia un componente analítico robusto, reproducible y validado empíricamente, completando la maduración técnica de la arquitectura SIEM propuesta.